A Size-Invariant Deep Reinforcement Learning Policy for the Interval Job Shop Scheduling Problem

Hernán Díaz Rodríguez[1*]

[1*]Department of Computing, University of Oviedo, Gijón, Spain.

Corresponding author(s). E-mail(s): diazhernan@uniovi.es;

Abstract

The interval job shop scheduling problem (IJSP) models uncertain processing times as closed intervals and is solved in the literature by per-instance metaheuristic search. This paper studies the complementary regime: a constructive policy, trained once by deep reinforcement learning, that returns a schedule in seconds. The policy combines dimensionless state features with a permutation-invariant encoder, so one parameter set covers any instance size. Trained with proximal policy optimization on four interval Taillard **20×15** instances, it attains **13.8%** mean relative error against crisp lower bounds on the validation instances of that size, compared with **27.9%** for the strongest hand-crafted rule, and outperforms every such rule on all seventy instances, including six size classes never trained on. A comparison with a dispatching rule evolved by genetic programming for the same benchmark shows the ranking of the two learning paradigms to be governed by the inference budget: the evolved rule prevails at a single pass, whereas the policy prevails on 46 of 70 instances when both draw **1024** samples. Ablation attributes the policy's performance to the interval training data rather than to interval-specific features, in agreement with the closed form of the objective: width features are removable at no cost, whereas training on crisp midpoints costs **0.63** points. Under Monte Carlo execution with durations drawn independently and uniformly within their intervals, its schedules track their predicted makespans more faithfully than those of every hand-crafted rule, and a width-penalizing variant carries its narrower predictions through to execution.

Keywords: job shop scheduling, interval uncertainty, deep reinforcement learning, neural combinatorial optimization, genetic programming hyper-heuristics, size invariance, dispatching

1 Introduction

The job shop scheduling problem (JSP) asks for a sequencing of a set of jobs, each of which visits the machines of a shop in its own prescribed order, that minimizes the overall completion time. The problem lies at the core of manufacturing operations planning: NP-hard (Garey, Johnson, & Sethi, 1976) and industrially ubiquitous, it is studied predominantly under the assumption that every processing time is known exactly. Real production environments seldom satisfy that assumption: schedules are committed before the work they plan is executed, and the processing times they assume are no more than estimates, displaced in practice by tool wear, operator variability and material differences. Among the formalisms proposed for this uncertainty, stochastic and fuzzy models (González-Rodríguez, Vela, & Puente, 2010) require the decision maker to supply probability distributions or membership functions; the *interval* model requires only bounds. In the resulting interval JSP (IJSP), each duration is a closed interval, schedule evaluation propagates those intervals, and the makespan is itself an interval, compared through a ranking criterion (Díaz, Palacios, González-Rodríguez, & Vela, 2023a; Lei, 2011).

The state of the art in IJSP makespan minimization is currently held by per-instance metaheuristic search: elitist artificial bee colony algorithms (Díaz et al., 2023a; Díaz, Palacios, González-Rodríguez, & Vela, 2023b) reach mean relative errors (RE, measured against crisp Taillard lower bounds) between roughly 6% and 16% depending on the size class. This cost is structural: the search must be repeated for every new instance, at seconds to minutes per run on dedicated hardware.

At the opposite end of the computational spectrum lie *constructive* methods, which build a schedule in a single pass without search. The classical representatives are priority dispatching rules, myopic functions such as shortest processing time or most operations remaining (Haupt, 1989; Panwalkar & Iskander, 1977), which are fast and interpretable but leave a wide quality gap on the IJSP. Genetic programming (GP) hyper-heuristics narrow that gap by evolving dispatching rules offline from a training set (Branke, Nguyen, Pickardt, & Zhang, 2016; Nguyen, Mei, & Zhang, 2017); a rule recently evolved for the interval benchmark studied here (Díaz Rodríguez, 2026b) constitutes the strongest constructive baseline available. Both families deliver schedules at negligible per-instance cost, which makes them the natural referents and competitors for a learned policy.

The deterministic scheduling literature has developed a third constructive approach, *learning to dispatch*: a deep reinforcement learning (DRL) agent builds the schedule constructively, choosing one eligible operation at a time (Park, Chun, Kim, Kim, & Park, 2021; Tassel, Gebser, & Schekotihin, 2021; Zhang et al., 2020). The training cost is incurred once; thereafter the policy returns a schedule for a new instance in seconds, which makes it attractive both as a standalone solver under tight time budgets and as an initializer for search. This paradigm has not, however, been extended to interval durations: a recent survey of learning-based job shop scheduling (Xu, Mei, Zhang, & Zhang, 2025) lists no method addressing interval-valued processing times.

Two obstacles separate standard JSP-DRL designs from the IJSP. The first is representational: the uncertainty must be observable to the agent, which should treat an operation with duration $[10, 11]$ differently from one with duration $[5, 16]$ even though both share the same midpoint. The second is architectural: the amortization argument that motivates learning requires one network to serve instances of any size. Designs that encode the instance as a fixed-dimension tensor are tied to the size class they were trained on, and graph-based encoders achieve size transfer (Park et al., 2021; Zhang et al., 2020) through message passing over the full disjunctive graph; whether the same invariance can be obtained with a substantially simpler encoder is a design question this paper answers in the affirmative.

This paper addresses both obstacles and asks three questions. *Q1:* can a constructive policy trained on a small number of interval instances compete with the strongest constructive alternatives (hand-crafted dispatching rules and a rule evolved by genetic programming), and where does it stand relative to the per-instance metaheuristic state of the art? *Q2:* can a single network, trained on one size class, remain useful across the whole benchmark without retraining? *Q3:* which standard components of JSP-DRL practice (best-of-N inference, attention-based encoders and automatic algorithm configuration) yield a measurable benefit at practical training budgets?

Contributions.

1. *A size-invariant, uncertainty-aware policy for the IJSP* (Section 4). All features are dimensionless: temporal quantities are scaled by a per-instance lower bound and interval uncertainty enters as relative widths. A Deep Sets (Zaheer et al., 2017) encoder over the eligible-operation set, combined with a pooled global context, yields policy *and* value networks of $\approx 1.2 \times 10^5$ parameters, independent of the numbers of jobs and machines.
2. *An empirical characterization* (Section 6) answering Q1–Q3: the scaling of solution quality with the inference budget on the training class, the zero-shot evaluation of a single network across all seven Taillard size classes, and its transfer to a second benchmark of classical instances.
3. *A comparison against both families of constructive heuristic* (Section 6.2), on the 70 benchmark instances and with per-instance results reported in full (Appendix A). The hand-crafted side comprises the traditional dispatching rules: SPT, LPT, MOR, MWKR and EST, plus two Giffler–Thompson variants. The learned-symbolic side is a rule evolved by genetic programming (GP) for this same benchmark (Díaz Rodríguez, 2026b), a comparison whose absence from the literature the survey of Xu et al. (2025) highlights.
4. *An analysis of what justifies its cost* (Section 7): a permutation audit of which inputs the network consults; five controlled ablations (inference budget, the attention variant, and three that delimit what interval awareness contributes); a λ-sweep of a width-penalizing training objective; and a Monte Carlo comparison of how faithfully the schedules execute.

Section 2 reviews related work and Section 3 states the problem. Section 4 presents the policy, Section 5 the experimental methodology and Section 6 the results; Section 7 analyses them and states the limitations, and Section 8 concludes.

2 Related Work

2.1 Job shop scheduling under interval uncertainty

Uncertain processing times have been modeled with probability distributions, fuzzy numbers and intervals. The fuzzy line is the most mature: durations are represented as fuzzy numbers, schedules are compared through the expected value of their fuzzy makespan, and two decades of population-based solvers have consolidated both the model and its evaluation conventions (Abdullah & Abdolrazzagh-Nezhad, 2014; González-Rodríguez et al., 2010; Palacios, González-Rodríguez, Vela, & Puente, 2015). The interval model can be read as the minimal-commitment limit of that representation: it retains only the support bounds, asks the decision maker for no shape information inside them, and is the standard minimal description of uncertainty in robust discrete optimization at large, where it underpins min–max and min–max-regret formulations (Kasperski & Zieliński, 2016; Kouvelis & Yu, 1997). Lei (2011) opened the interval JSP line, propagating interval durations through the schedule with interval arithmetic and steering a population-based neighborhood search by direct comparison of interval makespans. Later work broadened both the model and its toolkit. Díaz, González-Rodríguez, Palacios, Díaz, and Vela (2020) compared the candidate interval rankings systematically within a genetic algorithm and identified the lexicographic criterion adopted in this paper as the most robust choice for the problem; Díaz, Palacios, Díaz, Vela, and González-Rodríguez (2022) shifted the objective to total tardiness and paired it with explicit robustness measures; Sotskov, Matsveichuk, and Hatsura (2020) delimited analytically solvable special cases; and Zheng, Xiao, Zhang, and Lv (2024) carried interval durations into richer shop variants. On the Taillard-derived benchmark used here, the strongest published makespan results belong to the elitist artificial bee colony family (Díaz et al., 2023a, 2023b), whose successive refinements, elitist replacement and neighborhood filtering among them, progressively reduced the mean relative error to the single digits on most size classes; these works also fixed the evaluation conventions this paper adopts (interval arithmetic, lexicographic ranking during construction, expected makespan for reporting). Common to the whole line is per-instance search; fast constructive alternatives for the IJSP have begun to be explored only recently, through the evolved rule discussed in Section 2.3.

2.2 Priority dispatching rules

The oldest constructive approach to the job shop builds a schedule in a single left-to-right pass, repeatedly choosing which of the eligible operations to place next. A *priority dispatching rule* makes that choice by scoring the candidates with a fixed formula over their attributes. The rules this paper uses as baselines and as verification anchors are the classical single-attribute ones, retained here with their usual acronyms. Two look at the operation itself: *shortest* and *longest processing time* (SPT, LPT) prefer the smallest and the largest duration. One looks at the schedule built so far: *earliest starting time* (EST) prefers the operation that could begin soonest given the current occupation of its machine and the progress of its job. Two look at the job the operation belongs to: *most work remaining* and *most operations remaining* (MWKR, MOR) prefer the job with the most pending work, counted in time units and in operations respectively. Under interval durations these attributes are intervals themselves, and candidates are compared with the interval ranking introduced in Section 3. Surveys catalogue well over a hundred variants of such rules (Haupt, 1989; Panwalkar & Iskander, 1977) and recent ones revisit those still in use (Đurašević & Jakobović, 2018).

A refinement worth separating from the rules themselves is the active-schedule generator of Giffler and Thompson (1960), which at each step restricts the candidates to the conflict set of the machine that would finish earliest, and applies a priority rule only to break ties within that set. The restriction guarantees *active* schedules, in which no operation can be started earlier without delaying another, and improves markedly on the plain rules: on the benchmark used here it takes MWKR from 64.7% to 29.5% of RE. We write G&T-SPT and G&T-MWKR for the two combinations used here.

These rules require one pass and no search, which makes them the cheapest constructive option available. Their limitation is that a fixed formula cannot suit every shop and objective (Haupt, 1989), which motivates the learned alternatives of Sections 2.3 and 2.4, and is also why a constructive policy is compared against these rules and not against per-instance metaheuristics, which belong to a different computational class.

2.3 Genetic programming hyper-heuristics

Learning constructive heuristics is older than its neural incarnation. Genetic programming hyper-heuristics evolve priority expressions, trees over hand-crafted attributes such as processing time or work remaining, offline on a training set, and deploy the best expression as an ordinary dispatching rule: the artifact is interpretable and its inference cost is that of evaluating a formula. Design choices (attribute sets, fitness, generalization across instances) are reviewed by Branke et al. (2016), and Nguyen et al. (2017) organize the broader family under a unified framework. For the job shop specifically, Nguyen, Zhang, Johnston, and Tan (2013) showed that the choice of rule representation materially shapes the quality of the evolved dispatchers, and Karunakaran, Mei, Chen, and Zhang (2016) evolved rules for dynamic job shops with uncertain processing times, exposing duration deviations to the rules as attributes, under an uncertainty model that is, once again, distributional rather than interval-valued. A recent study (Díaz Rodríguez, 2026b) develops the GP side for the interval benchmark used here, evolving interval-aware rules under the same ranking and evaluation conventions, with both a deterministic pass and a sampling variant.

2.4 Neural constructive policies for scheduling

Neural combinatorial optimization began with pointer networks (Vinyals, Fortunato, & Jaitly, 2015), sequence models that emit a permutation of their input and can therefore represent tours and schedules; Bello, Pham, Le, Norouzi, and Bengio (2016) trained them with policy gradients, removing the need for supervised targets, and attention-based encoder–decoders trained against greedy-rollout baselines then became the standard for routing (Kool, van Hoof, & Welling, 2019; Nazari, Oroojlooy, Snyder, & Takáč, 2018). Reinforcement learning for job-shop dispatching predates the deep era: Gabel and Riedmiller (2008) already learned decentralized dispatching policies by gradient ascent. Its modern applications to machine scheduling are surveyed by Kayhan and Yildiz (2023). For the deterministic JSP the reference design is that of Zhang et al. (2020): the partial schedule is encoded as a disjunctive graph, a graph neural network embeds it, and the policy trained by PPO on small synthetic instances generalizes zero-shot to larger ones while surpassing the classical priority rules. Park et al. (2021) refine that representation and document the cross-size generalization explicitly, and Tassel et al. (2021) show that the environment formulation itself (state design and reward shaping) accounts for a substantial share of the attainable performance. The paradigm has since expanded along three axes: to richer shops, with heterogeneous-graph policies for the flexible JSP (Song, Chen, Li, & Cao, 2023); from construction to improvement, with DRL-guided local search over complete schedules (Zhang, Cao, Song, Wu, & Zhang, 2024); and toward uncertainty. On that third axis, Grumbach, Müller, Reusch, and Trojahn (2024) size slack buffers in stochastic flow shops under a distributional, event-driven model; Smit, Wu, Troubil, Zhang, and Nuijten (2024) extend constructive policies to stochastic processing times by processing sampled duration scenarios with an attention module; and Wang, Guo, and Liu (2025) train an autoregressive generative model for the fuzzy job shop. The interval representation, the minimal of these uncertainty models, remains unaddressed by this literature, which is the gap the present paper fills.

Two further components of the toolbox matter here. Sampling several rollouts at inference and keeping the best is a standard quality lever whose returns grow with the diversity of the learned distribution (Kool et al., 2019; Kwon et al., 2020). Permutation-invariant set encoders were formalized as Deep Sets (Zaheer et al., 2017), of which self-attention is the natural strict generalization; the architecture of Section 4.3 builds on the former and evaluates the latter as a controlled ablation.

The survey by Xu et al. (2025) compares the GP and RL families for job shop scheduling and notes the scarcity of comparisons under a common protocol. The availability of the evolved rule of Section 2.3 on the very benchmark studied here lets Section 6.2 put the two learned artifacts on the same instances at matched *inference* budgets.

3 Problem Statement

Definition 1 (IJSP) An IJSP instance comprises n jobs $J_1, \dots, J_n$ and m machines, indexed $1, \dots, m$. Job J_j is a fixed sequence of m operations $(o_{j1}, \dots, o_{jm})$, where operation o_{jk} requires machine $\nu_{jk} \in \{1, \dots, m\}$ and an uncertain processing time known only to lie in the interval $\mathbf{p}_{jk} = [p^L_{jk}, p^U_{jk}]$, with $0 \leq p^L_{jk} \leq p^U_{jk}$. Each job visits each machine exactly once, so $(\nu_{j1}, \dots, \nu_{jm})$ is a permutation of the machine indices. Each machine processes at most one operation at a time, operations of the same job cannot overlap, and preemption is not allowed.

A solution is a processing order of all nm operations, that is, a sequence in which o_{jk} appears before $o_{j,k+1}$ for every job j and every $k < m$. We encode it as a *permutation with repetition* of job indices, where the k-th occurrence of j denotes o_{jk}. The order induces, on each machine, a sequence over the operations that require it, and its *semiactive schedule* starts every operation as early as those sequences permit: no operation can start before its job predecessor $JP(o_{jk}) = o_{j,k-1}$ or its machine predecessor $MP(o_{jk})$, the operation immediately preceding o_{jk} on machine ν_{jk} in the induced sequence, have completed. With component-wise interval addition $[a, b] + [c, d] = [a + c, b + d]$ and join $\max([a, b], [c, d]) = [\max(a, c), \max(b, d)]$, the start and completion times of every operation are themselves intervals, given by the recursion

$$\mathbf{s}_o = \max\big(\mathbf{c}_{JP(o)}, \mathbf{c}_{MP(o)}\big), \qquad \mathbf{c}_o = \mathbf{s}_o + \mathbf{p}_o, \tag{1}$$

When a predecessor does not exist (the job predecessor of a job's first operation, or the machine predecessor of the first operation scheduled on a machine), its completion is taken as $[0, 0]$, so that the corresponding argument is neutral in the max and the start is decided by the predecessor that does exist. The interval makespan collects the job completions component-wise,

$$\mathbf{C}_{\max} = \Big[\max_j c^L_{o_{jm}},\ \max_j c^U_{o_{jm}}\Big] =: \big[C^L_{\max}, C^U_{\max}\big]. \tag{2}$$

A *realization* of the instance assigns to each operation a crisp duration $p_{jk} \in \mathbf{p}_{jk}$; *executing* a processing order under a realization means evaluating the recursion of Eq. (1) with those crisp durations, which yields a crisp makespan. Because the recursion is monotone in the durations, the executed makespan of *every* realization falls inside $\mathbf{C}_{\max}$. Intervals carry no canonical total order, so schedule comparison requires a ranking choice; during construction and search we use the lexicographic order

$$\mathbf{a} \prec_{lex} \mathbf{b} \iff a^U < b^U \ \vee\ \big(a^U = b^U \wedge a^L < b^L\big) \tag{3}$$

for intervals $\mathbf{a} = [a^L, a^U]$ and $\mathbf{b} = [b^L, b^U]$, which prioritizes the worst case, in line with min–max robust optimization (Bertsimas & Sim, 2004; Kasperski & Zieliński, 2016). The optimization problem is then to find a processing order whose interval makespan is minimal under this order. This modeling choice is not reopened here: a systematic study of the alternative ranking criteria found the lexicographic order the most robust for this problem (Díaz et al., 2020), and it is kept fixed throughout so that every method in this paper is judged by the same criterion. For reporting we follow the field convention (Díaz et al., 2020, 2023a):

$$\mathrm{RE}(\%) = \frac{E[\mathbf{C}_{\max}] - \mathrm{LB}}{\mathrm{LB}} \times 100, \qquad E[\mathbf{C}_{\max}] = \tfrac{1}{2}\big(C^L_{\max} + C^U_{\max}\big), \tag{4}$$

with LB the published lower bound of the crisp Taillard counterpart (Taillard, 1993), an indicative, hardware-independent yardstick rather than a strict optimality gap, since the crisp JSP and the IJSP are different problems.

Remark 1 For instances whose intervals are symmetric perturbations of a crisp instance, the midpoint scenario coincides with the original crisp durations; comparisons under Eq. (4) are therefore scenario-consistent (midpoint solution quality against the midpoint-scenario bound).

4 Method

Reinforcement learning is learning to decide from experience. An *agent* interacts with an environment in steps: it observes the current *state* s, chooses an *action* a among those available, and receives a numeric *reward* signalling how good the consequences were. The agent's behavior is its *policy* $\pi(a \mid s)$: a function that, given the state s, assigns a probability to each available action a. The policy is the object of learning: training increases the probability of actions followed by a high cumulative reward (the *return*).

Policy-gradient methods such as PPO (Schulman, Wolski, Dhariwal, Radford, & Klimov, 2017) judge an action not by its return alone but by comparing that return with what was *expected* from the state where the action was taken; only the surplus, the *advantage*, moves the policy. The expectation is itself learned: a *value function* $V(s)$ estimates the return attainable from state s. In practice both functions share one network with two outputs, a *policy head* that scores the available actions and a *value head* that estimates $V(s)$, trained jointly. This is why the network in this paper has two heads.

Instantiated on scheduling, the agent is a dispatcher and schedule construction is a finite-horizon *Markov decision process* of exactly nm steps, one per operation. The state after t steps is the partial schedule (each job's next-operation pointer, the job and machine completion intervals), a sufficient statistic for the remaining decisions; the available actions are the *eligible* operations, that is, the next unscheduled operation of each job, collected in the set $\mathcal{E}_t$ with $1 \leq |\mathcal{E}_t| \leq n$ and written $\mathcal{E}$ when the step is immaterial. The generic $\pi(a \mid s)$ thus specializes to $\pi(i \mid s)$, a distribution over which eligible operation i to schedule next; the transition applies Eq. (1) deterministically; an *episode* is the construction of one complete schedule, and the reward is a function of the makespan achieved. The trained policy is then a dispatching criterion in its own right: at each step it orders the eligible operations by preferences learned from experience rather than by a fixed, hand-written formula.

Figure 1 shows the network, read from the input at the top down to the two outputs. Each eligible operation enters as a row of 16 numbers that describe it: how long it may take, how much work its job still carries, how soon it could start (Section 4.2). A multilayer perceptron (MLP), the *same* one for every candidate, turns each row into an *embedding*: a vector of h learned quantities that re-describe the candidate.

The number of candidates varies from step to step, so the embeddings are *pooled*: their mean and their componentwise maximum are two fixed-size summaries of the whole set. Joined with 12 numbers describing the shop as a whole (the *global state*), these summaries are condensed by another MLP into a single *context* vector g, a fixed-size summary of the current decision state.

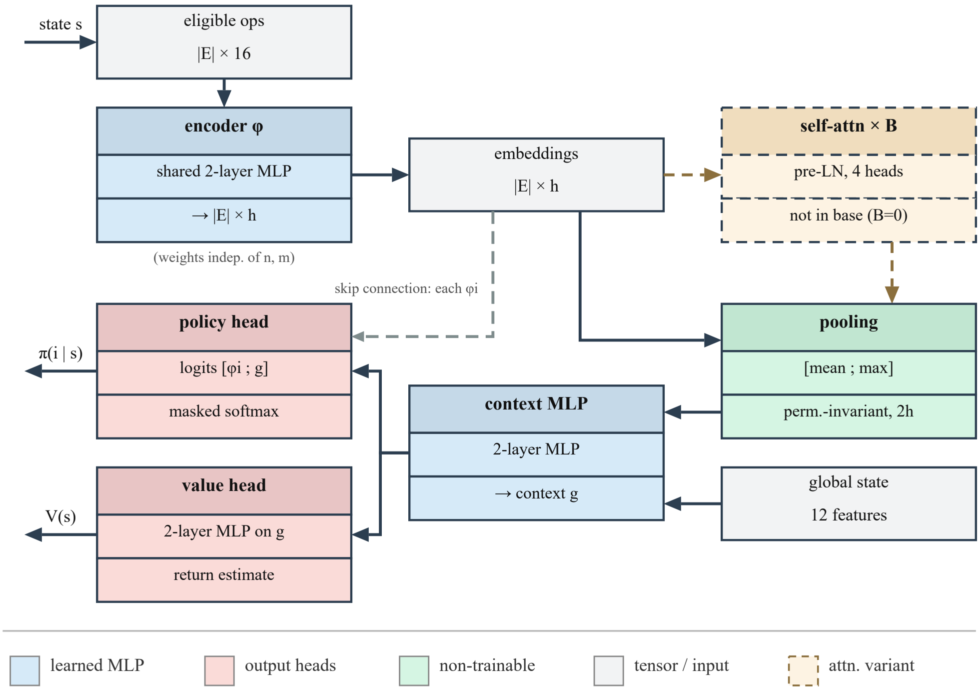

Fig. 1 The size-invariant policy/value network. Solid edges are the base model's data path; the grey dashed edge is the per-candidate skip connection; the dashed detour is the self-attention variant of Section 4.4, not part of the base model ($B{=}0$).

The two heads introduced above read from this context. Pooling, however, discards by design which candidate is which, so the context alone cannot rank them. The policy head therefore receives two inputs, the shared context g and each candidate's own embedding, carried past the pooling stage by the *skip connection* visible in Figure 1. The head is applied once per candidate: for eligible operation i it reads the pair (embedding of i, context) and outputs a single scalar score, so $|\mathcal{E}|$ candidates yield $|\mathcal{E}|$ scores, each tied to its operation by construction because it was computed from that operation's embedding. A *softmax* over these scores (the *logits*, in network terms) turns them into the probabilities $\pi(i \mid s)$, one per eligible operation, however many there are (*masked* means that unused padding slots are excluded). The value head estimates the situation rather than any one candidate, so it reads g alone and outputs $V(s)$. In total, four two-layer MLPs (the candidate encoder ϕ, the context combiner, and the two heads) are composed into a single network and trained jointly end to end.

The dashed detour between the embeddings and the pooling is an *optional* self-attention stage, evaluated as a variant in Section 4.4; the base model, which every headline result of this paper uses, omits it.

Sections 4.1–4.4 specify, in order, the reward, the 16+12 input quantities, the network stages, and the self-attention variant. Training and inference are described with the experimental setting in Section 5.4.

4.1 The reward

The *reward* at step t is a weighted sum of six shaping components,

$$r_t \;=\; w_{\mathrm{mk}}\, r_t^{\mathrm{mk}} + w_{\mathrm{pr}}\, r_t^{\mathrm{pr}} + w_{\mathrm{id}}\, r_t^{\mathrm{id}} + w_{\mathrm{li}}\, r_t^{\mathrm{li}} + w_{\mathrm{cr}}\, r_t^{\mathrm{cr}} + w_{\mathrm{ba}}\, r_t^{\mathrm{ba}}, \tag{5}$$

where each $r_t^{\bullet}$ is one shaping component and $w_\bullet \in [0,1]$ its fixed weight: terminal makespan (mk), job progress (pr), machine idleness (id), local improvement (li), operation criticality (cr) and load balance (ba), each defined below.

Each component is divided by a magnitude of the instance, listed with it, which makes the reward dimensionless and comparable across instances and sizes. Several scales use $\widehat{\mathrm{LB}}$, a lower bound on the makespan that the method computes from the instance itself as the larger of the two classical relaxations, the most loaded machine and the longest job, in worst-case durations:

$$\widehat{\mathrm{LB}} = \max\Big(\max_{\mu} \sum_{(j,k)\,:\,\nu_{jk}=\mu} p^U_{jk},\;\; \max_j \sum_{k=1}^{m} p^U_{jk}\Big).$$

It is an internal estimate, so the method does not depend on externally supplied references. In the definitions below, write c^U_μ for the worst-case completion time of machine μ over the operations scheduled so far, $M^U_t = \max_\mu c^U_\mu$ for the projected worst-case makespan after t decisions, and let step t schedule operation o_{jk} on machine $\mu = \nu_{jk}$. The six components are:

- *Terminal makespan* (w_{mk}=1.0): zero at every step except the last, where

  $$r^{\mathrm{mk}}_{nm} = -\frac{C^U_{\max}}{s_{\mathrm{mk}}}, \qquad s_{\mathrm{mk}} = \frac{\widehat{\mathrm{LB}}}{10},$$

  Substituting the scale gives $r^{\mathrm{mk}}_{nm} = -10\,(1+g)$, with $g = (C^U_{\max} - \widehat{\mathrm{LB}})/\widehat{\mathrm{LB}}$ the relative gap to the instance bound: a function of the relative gap alone, so its magnitude is independent of the instance's size and time scale, and a schedule of the same relative quality receives the same reward on any instance. The robust arms of Section 7.4 replace $C^U_{\max}$ here by f_λ.
- *Job progress* (w_{pr}=0.26):

  $$r^{\mathrm{pr}}_t = \frac{1}{nm}.$$

  A constant per operation scheduled, a uniform progress signal that sums to one over any episode.

- *Machine idleness* ($w_{\text{id}}\!\approx\!0.24$):

$$r_t^{\text{id}} = -\frac{\max\left(0,\, s^U_{o_{jk}} - c^U_\mu\right)}{s_{\text{id}}}, \qquad s_{\text{id}} = 2\bar{p},$$

  with $\bar{p}$ the mean midpoint duration of the instance: the worst-case idle gap that scheduling o_{jk} opens on its machine, measured in units of typical operations.
- *Local improvement* ($w_{\text{li}}\!=\!0.15$):

$$r_t^{\text{li}} = \kappa_t\,\frac{M^U_{t-1} - M^U_t}{s_{\text{mk}}/10}, \qquad \kappa_t = \begin{cases} 2 & M^U_t > M^U_{t-1}, \\ 1 & \text{otherwise}, \end{cases}$$

  the step change of the projected makespan, with deteriorations penalized twice as strongly: a dense signal so that early decisions are not credited only nm steps later.
- *Operation criticality* ($w_{\text{cr}}\!=\!0.1$):

$$r_t^{\text{cr}} = \frac{\sum_{k'>k} p^U_{jk'}}{s_{\text{cr}}}, \qquad s_{\text{cr}} = \frac{W^U}{2n},$$

  with W^U the instance's total worst-case work: the worst-case work remaining in the scheduled operation's job. Favoring jobs with much work remaining is a standard proxy for the emerging critical path.
- *Load balance* ($w_{\text{ba}}\!=\!0.1$):

$$r_t^{\text{ba}} = -\frac{\sigma_\mu\left(c^U_\mu\right)}{s_{\text{ba}}}, \qquad s_{\text{ba}} = 1.5\,\sigma_\mu\left(L^U_\mu\right),$$

  where σ_μ denotes the standard deviation across the m machines, taken after the step over the worst-case completion times and, in the scale, over the total worst-case loads $L^U_\mu = \sum_{(j,k)\,:\,\nu_{jk}=\mu} p^U_{jk}$ of the instance. Referring the observed dispersion to the dispersion the instance's own loads exhibit discourages lopsided machine loads without penalizing an instance for being intrinsically unbalanced.

The weights above and the internal constants of the components were all fixed by hand during early development and carried unchanged into every experiment of this paper. No claim of optimality is attached to them. The configuration campaign of Section 5.3 later raced the six weights, each free in $[0, 1]$, and did not improve on the hand-set vector within the protocol's resolution; the internal constants were not raced.

4.2 State representation

The state observed at each decision comprises two groups of features: a per-operation description of every eligible operation, scored one candidate at a time by the policy head, and a description of the shop as a whole, which enters the shared context. Operation $o = o_{jk}$ on machine $\mu = \nu_{jk}$ is described by the 16 quantities of Table 1, and the global state by the 12 aggregates of Table 2.

Both groups are dimensionless: every temporal quantity is expressed as a fraction of the computed bound $\widehat{\mathrm{LB}}$ of Section 4.1, and every uncertainty as a ratio of interval width to magnitude. Absolute times are not comparable across instances, since their magnitude reflects the time scale of the instance at hand, and a policy trained on them would be tied to the scale of its training set. The dimensionless representation renders the observation comparable across instances and sizes, which is one of the two components of size invariance; the other is the architecture of Section 4.3.

Table 1 Per-operation features (all dimensionless). $d = \mathbf{p}_{jk}$, $\mathbf{est}$ = earliest start interval, $\mathbf{rem}$ = remaining work interval of the job strictly after o, excluding o itself; $\mathrm{mid}(\cdot)$ is the interval midpoint; $L(\mu)$ the remaining worst-case load of machine μ; $\bar{L}$ its average over machines; $\mathbf{c}_J$ and $\mathbf{c}_\mu$ the completion intervals of the operation's job and machine over what is scheduled so far.

#	Feature	Definition
1–2	duration bounds	$d^L/\widehat{\mathrm{LB}}$, $d^U/\widehat{\mathrm{LB}}$
3	duration uncertainty	$(d^U - d^L)/\mathrm{mid}(d)$
4–5	earliest start bounds	$est^L/\widehat{\mathrm{LB}}$, $est^U/\widehat{\mathrm{LB}}$
6	start uncertainty	$(est^U - est^L)/\mathrm{mid}(\mathbf{est})$
7–8	remaining job work	$rem^L/\widehat{\mathrm{LB}}$, $rem^U/\widehat{\mathrm{LB}}$
9–10	job position	$(m-k-1)/m$, k/m
11	machine load	$L(\mu)/\widehat{\mathrm{LB}}$
12	potential idle gap	$\max(0,\, c_J^U - c_\mu^U)/\widehat{\mathrm{LB}}$
13	machine congestion	$L(\mu)/\bar{L}$
14	slack	$(est^U - \min_{o' \in \mathcal{E}} est_{o'}^U)/\widehat{\mathrm{LB}}$
15–16	completions	$c_\mu^U/\widehat{\mathrm{LB}}$, $c_J^U/\widehat{\mathrm{LB}}$

Table 2 Global state features (all dimensionless aggregates, notation as in Table 1; no dimension depends on n or m). Rows that list several statistics contribute one feature each, for 12 in total.

#	Feature	Definition
1	progress	scheduled operations / nm
2	partial makespan	$\max_\mu c_\mu^U/\widehat{\mathrm{LB}}$
3–4	machine completions	mean and std of $c_\mu^U/\widehat{\mathrm{LB}}$
5–7	remaining loads	mean, max and std of $L(\mu)/\widehat{\mathrm{LB}}$
8–9	remaining job work	mean and max of job remaining $/\widehat{\mathrm{LB}}$
10	eligibility	$\lvert\mathcal{E}\rvert/n$
11	pending uncertainty	$\sum$(widths)/$\sum$(midpoints) of unscheduled ops
12	total load	$\sum_\mu L(\mu)/(\widehat{\mathrm{LB}} \cdot m)$

Every quantity consulted by the dispatching rules of Section 2.2 (processing times, remaining work, earliest start times, slack, machine load) appears among the per-operation features, so that no advantage over those rules can be attributed to a wider per-operation vocabulary. The representation extends that shared vocabulary in two directions: the interval bounds and relative widths of every temporal quantity, which make the uncertainty explicit, and the global aggregates, which no per-operation priority rule consults. Which of these inputs the trained network comes to rely on is measured a posteriori by the audit of Section 7.1.

4.3 Size-invariant architecture

The second component of size invariance is architectural. The number of candidates changes at every step and from instance to instance, while a neural network requires inputs of fixed dimension. Two elements resolve this: the *same* network reads every candidate, so any number of them can be processed, and the resulting embeddings are *pooled* into fixed-size summaries, so the scoring stage receives a representation of the whole decision. Let $x_i \in \mathbb{R}^{16}$ be the features of eligible operation i and $g_0 \in \mathbb{R}^{12}$ the global state; the construction has three stages (Figure 1).

Embed. The shared two-layer MLP ϕ (width h=128, LayerNorm, ReLU, orthogonal initialization) maps each candidate to an embedding $\phi_i = \phi(x_i)$. The whole eligible set is processed in one forward pass, as the $|\mathcal{E}| \times 16$ matrix of its rows. Sharing the weights across rows allows the stage to accept any number of candidates and makes it equivariant to their order. Candidates do not interact at this stage: ϕ_i depends on x_i alone.

Pool into a context. The variable-size set $\{\phi_1, \ldots, \phi_{|\mathcal{E}|}\}$ must be reduced to a fixed-size vector before it can be combined with the global state. Mean and max pooling provide two complementary permutation-invariant summaries (Zaheer et al., 2017) which, concatenated with the global state g_0, are mixed into the context:

$$g = \mathrm{MLP}_{ctx}\left(\left[\tfrac{1}{|\mathcal{E}|}\textstyle\sum_i \phi_i;\ \max_i \phi_i;\ g_0\right]\right) \in \mathbb{R}^h. \tag{6}$$

Pooling introduces no parameters: it maps the $|\mathcal{E}| \times h$ embeddings to the $2h$ summaries the context MLP consumes. Both statistics are taken over the non-padded candidates only, since in batched training the sets are padded to the batch maximum: the mean divides by the true count and the max ignores the padded rows. This embed-then-pool pattern is the *Deep Sets* construction of Zaheer et al. (2017), whose central result is that, for suitable choices of the two maps, any permutation-invariant function of a set admits this form. What the pattern restricts is therefore the invariance and not the class of functions; what the finite network used here represents remains bounded by its width. The result tables refer to the base model by this name, in contrast to its attention variant (Section 4.4).

Score. The *policy head* scores each pair (candidate embedding, context), and a masked softmax over the scores yields the distribution over eligible operations. The *value head* computes the state value $V(s)$ used in the advantage estimation, reading the context alone, since that expectation concerns the state rather than any one candidate:

$$\pi(i \mid s) = \operatorname{softmax}_i \operatorname{MLP}_\pi([\phi_i; g]), \qquad V(s) = \operatorname{MLP}_V(g). \tag{7}$$

Table 3 specifies the four trainable blocks. Each is a two-layer perceptron with LayerNorm and ReLU between its layers and a linear output; all hidden layers are initialized orthogonally with gain $\sqrt{2}$ and zero biases. The output layer of the policy head uses gain 10^{-2}, so at the start every candidate receives a nearly equal score and the untrained policy is close to uniform; the output layer of the value head uses gain 1. Pooling and the masked softmax carry no parameters, so those four blocks account for the whole network: 120,322 parameters, no dimension of which depends on n or m.

Table 3 Layer specification of the policy/value network at the hidden width h=128. Shapes are input, hidden and output dimensions.

Block	Shape	Parameters
Candidate encoder ϕ	$16 \to 128 \to 128$	18,944
Pooling (mean, max)	$\lvert\mathcal{E}\rvert \times 128 \to 256$	—
Context MLP	$268 \to 128 \to 128$	51,200
Policy head	$256 \to 128 \to 1$	33,281
Value head	$128 \to 128 \to 1$	16,897
Base network (B=0)		**120,322**
Attention block (each)	128, 4 heads of 32; feed-forward $128 \to 256 \to 128$	132,480

4.4 The attention variant

Pooling summarizes the candidate set by two statistics, against which every candidate is then scored. This construction cannot express a comparison between two *particular* candidates: whether operation i should take the machine now may depend on which other eligible operation would occupy it next, and the mean embedding does not identify that candidate. The embeddings of all $\lvert\mathcal{E}\rvert$ candidates are available at the decision step, so what the architecture lacks is not the information but a path along which one candidate can condition another. *Self-attention* (Vaswani et al., 2017) supplies such a path and is the component most commonly adopted for this purpose in the neural combinatorial optimization literature (Section 2.4). In an attention layer every candidate emits a *query*, a *key* and a *value*; the similarity between one candidate's query and another's key decides how much of that other's value is added into the first's embedding. Each candidate is thereby rewritten as a content-weighted reading of all the others, with the weights computed from the embeddings themselves rather than fixed in advance.

Self-attention is evaluated here as a variant rather than adopted as part of the proposed model. A stack of B pre-layer-normalization (pre-LN) transformer blocks transforms the embeddings between the embedding and pooling stages of Figure 1. Writing $\Phi \in \mathbb{R}^{|\mathcal{E}|\times h}$ for the stacked embeddings, each block computes

$$Z = \Phi + \mathrm{MHA}\big(\mathrm{LN}(\Phi)\big), \qquad \Phi' = Z + \mathrm{FFN}\big(\mathrm{LN}(Z)\big), \tag{8}$$

where MHA is multi-head scaled dot-product attention (4 heads of dimension 32, no dropout), FFN is a two-layer perceptron ($128 \to 256 \to 128$, ReLU) applied to each row, and LN is LayerNorm. Both additions are *residual*, that is, each block adds to its input rather than replacing it, so an untrained block is close to the identity and the stack starts near the base model; the pre-LN arrangement is preferred over the original post-LN one for this property (Xiong et al., 2020). No positional encoding is used, which keeps self-attention permutation-equivariant, so the pooled context remains permutation-invariant and the size invariance of Section 4.3 is preserved. Padding slots are masked out of the attention weights.

The variant is a configuration option on the same code path, and $B{=}0$ reproduces the base network exactly, so the comparison of Section 7.3 differs in this component and in nothing else. A single block carries 132,480 parameters in the query, key and value projections, the output projection, two layer normalizations and the feed-forward network (Table 3), so the $B{=}2$ variant evaluated here grows the network from 1.2×10^5 to 3.9×10^5 parameters, a factor of 3.2, and multiplies the wall-clock cost of an episode by 2.2 (Section 7.3).

During batched training, variable-size candidate sets are padded to the batch maximum with a boolean mask applied to attention, pooling and logits; unit tests verify that masked-padded forward passes are numerically identical to unpadded ones.

5 Experimental Methodology

5.1 Instances, data split and metric

Experiments use the 70 interval Taillard instances TA1–TA70 that anchor the recent IJSP literature (Díaz et al., 2023a, 2023b): seven size classes from 15×15 to 50×20, each crisp duration replaced by an integer interval symmetric around it, with half-width drawn uniformly up to 15% of the original value (Díaz et al., 2020). The benchmark is openly available (Díaz Rodríguez, 2026a). Of the 70 instances, the final model is trained on four, TA11–TA14 of the 20×15 class. Six more of the same class, TA15–TA20, form the *validation set*: never used for gradients, but every design and configuration decision was judged on their performance, so they are reported separately and excluded from claims about unseen data. The remaining 60 instances entered neither training nor validation. A second benchmark, consisting of interval versions of twelve classical instances (FT10, FT20, La21, La24, La25, La27, La29, La38, La40 and ABZ7–9) generated with the same symmetric scheme (Díaz et al., 2020), is reserved for the cross-family evaluation of Section 6.3, and plays no part in training or validation either.

Table 4 Machine and software used for every experiment in this paper.

Component	Specification
CPU	AMD Ryzen 5 3400G, 4 cores / 8 threads, 3.7 GHz
Memory	32 GB
GPU	NVIDIA GeForce RTX 5060 Ti, 16 GB (tuning only)
Operating system	Windows 10 Pro 22H2 (64-bit)
Language	Python 3.12.6
Learning	PyTorch 2.9.1, CUDA 13.0 build
Numerics	NumPy 2.3.5, SciPy 1.17.0
Figures	Matplotlib 3.10.8; svglib 2.1.0 (diagram)
Tuning	irace 4.4.3 on R 4.6.1 (Section 5.3)

Table 5 Hyperparameters, identical across every experiment reported in this paper.

Hyperparameter	Value	Hyperparameter	Value
Optimizer		*PPO objective*	
learning rate (Adam)	$3 \cdot 10^{-4}$	clipping range	0.2
Adam ϵ	10^{-5}	GAE λ	0.95
gradient clipping	0.5	discount factor γ	1.0
Network and schedule		*PPO updates*	
hidden width h	128	epochs per update	4
weight initialization	orthogonal	minibatch size	256
policy update	every 4 ep.	entropy coefficient	0.01
greedy evaluation	every 10 ep.	advantage normalization	per update

5.2 Computational environment

Table 4 lists the machine and the software stack. One aspect bears on the costs reported later: every number in this paper (training, evaluation, the dispatching-rule baselines and the ablations) was produced on the CPU, with the single exception of the tuning campaign of Section 5.3. The network is small enough (Section 4.3) that a forward pass is negligible next to the environment step that follows it, so the GPU yields no speedup for a single rollout; it was used only for the vectorized batched rollouts of the operating-fidelity tuning campaign (Section 5.3), where many episodes advance in lockstep and the batched forward pass amortizes. The method therefore carries no dependence on specialized hardware.

5.3 Hyperparameter configuration

All reported results use the hyperparameters of Table 5. Their values were fixed by hand during development, most of them at the defaults customary in the PPO literature. To assess whether automatic tuning would improve on them, two configuration

Table 6 Search space of the two irace campaigns. Real parameters were sampled on a logarithmic scale, the rest from the listed values.

Parameter	Default	Campaign 1 (330 exp.)	Campaign 2 (300 exp.)
learning rate	$3 \cdot 10^{-4}$	$[10^{-4}, 10^{-3}]$	$[10^{-4}, 10^{-3}]$
entropy coefficient	0.01	[0.003, 0.03]	[0.003, 0.03]
clipping range	0.2	0.1, 0.2, 0.3	0.1, 0.2, 0.3
epochs per update	4	2, 4, 8	4, 8
GAE λ	0.95	0.90, 0.95, 0.98	0.90, 0.95, 0.98
hidden width	128	64, 128, 256	128, 256
minibatch size	256	128, 256, 512	fixed
update period	4 ep.	2, 4, 8 ep.	fixed

studies were run with irace (López-Ibáñez, Dubois-Lacoste, Pérez Cáceres, Birattari, & Stützle, 2016). The first raced 330 experiments (309 used) over the 8-parameter space of Table 6 at reduced fidelity: one training instance, 300 episodes, $\sim$8 minutes per evaluation. Its elites agreed on a more aggressive update regime and kept minibatch size and update period at their defaults, but at the full 1000-episode operating budget the best of them scored 14.5% mean RE against 13.4% for the defaults: the low-fidelity optimum did not transfer.

The second campaign raced 295 experiments at operating fidelity, with 1000 episodes on two instances per evaluation, over the reduced 6-parameter space of Table 6, seeded with the defaults and the first campaign's elites. It converged to a region distinct from the defaults and eliminated the seeded default during racing; yet its winner, retrained on the four training instances over three seeds, again failed to improve: 14.73% against 13.52% over the six validation instances. A third campaign raced the reward instead of the optimizer: the six weights of Eq. (5), each free in $[0, 1]$, over 299 experiments at operating fidelity. Its winner did not improve the deployed weights either, 14.04% against 13.55% under the same confirmation protocol (Wilcoxon signed-rank, $p = 0.21$). Two by-products of that campaign are more informative than its verdict. A seeded terminal-only vector $(1, 0, 0, 0, 0, 0)$ was eliminated in the first round, so the dense shaping is not dispensable; and four elites survived the full race differing by as much as 0.48 in a single weight, with a Kendall agreement across seeds of $W = 0.02$–0.05 on which is better, so at this budget the objective is flat in the reward weights relative to the noise between training runs.

Automatic configuration therefore does not improve on the values of Table 5 for this problem, even at the operating budget, nor do raced reward weights improve on the hand-set ones, within the resolution of the protocol. That resolution can be quantified: with a standard deviation of 0.47 points across seed means at 1000 episodes (Section 6.1) and three seeds per arm, the minimum difference a paired confirmation detects at conventional power is approximately one point of RE. The three campaigns therefore exclude improvements larger than that magnitude, not the existence of smaller ones. The same arithmetic explains their common signature, the racing optimum never matching the confirmation optimum: single-run outcomes are too noisy to

Algorithm 1 Best-of-N inference

1: $(\sigma^*, \mathbf{C}^*_{\max}) \leftarrow$ greedy rollout of π
2: **for** $i = 1$ **to** $N - 1$ **do**
3: $\quad(\sigma, \mathbf{C}_{\max}) \leftarrow$ sampled rollout of π
4: $\quad$**if** $\mathbf{C}_{\max} \prec_{lex} \mathbf{C}^*_{\max}$ **then** $(\sigma^*, \mathbf{C}^*_{\max}) \leftarrow (\sigma, \mathbf{C}_{\max})$
5: $\quad$**end if**
6: **end for**
7: **return** σ^*

separate near-equivalent configurations, and raising the resolution by repeating each evaluation reduces the number of configurations a fixed budget can race.

5.4 Training and inference

The policy is trained with PPO (Schulman et al., 2017) under the hyperparameters of Table 5. Training on a set of instances proceeds in blocks, one instance at a time, with the weights carried over from each block to the next. A *rollout* is one complete construction episode played by the current policy: *sampled* when each operation is drawn from π, as during training, and *greedy* when every step takes the most probable operation. Every tenth episode (Table 5), the current policy is additionally evaluated with one greedy rollout. Whenever an episode of either kind improves on the best makespan found so far, its solution and a copy of the weights that produced it are stored; the stored weights are the *checkpoint* used at evaluation time.

The discount factor is $\gamma = 1$ rather than the 0.99 customary in deep RL (Schulman et al., 2017). The horizon here is finite and known, so future rewards can be credited in full; at $\gamma = 0.99$ the terminal makespan, the quantity being minimized, would reach the earliest decisions of a 20×15 instance attenuated to $0.99^{nm} \approx 0.05$ of itself. Updates are likewise applied over transitions pooled from four consecutive episodes rather than from one, in minibatches of 256 and with advantages normalized within each update, because the nm steps of a single episode are few and strongly correlated: pooling widens the sample the advantage estimate is computed from, and normalizing keeps the gradient scale comparable across instances of different size.

Best-of-N inference.

At evaluation time the stochastic policy is exploited by drawing one greedy rollout and $N - 1$ sampled ones, and keeping the best solution under the ranking of Eq. (3) (Algorithm 1). The deployed budget is N=64; results are additionally reported for a single greedy pass and, in the budget-matched comparisons of Sections 6.1 and 6.2, for N=1024. This choice is motivated empirically in Section 7.2: sampling overtakes the greedy pass at twelve rollouts and continues to improve to the largest budget measured, so the policy is more valuable as a *distribution* than as its argmax, consistent with sampling-based decoding in neural combinatorial optimization (Kool et al., 2019).

5.5 Baselines

Constructive baselines are the dispatching rules of Section 2.2 (SPT, LPT, MOR, MWKR and EST) and the two Giffler–Thompson combinations. The results of Section 6 carry the strongest of each family, namely MOR, EST and G&T-MWKR, while SPT, LPT, MWKR and the SPT tie-break of Giffler–Thompson are dominated by a wide margin in every size class and are not reported further. The learned-symbolic baseline is the dispatching rule evolved by genetic programming for this same benchmark by Díaz Rodríguez (2026b), compared budget by budget in Section 6.2.

Three published IJSP metaheuristics are included as reference yardsticks rather than as direct competitors, since each searches every instance separately for minutes on dedicated hardware and reports 30-run averages: the genetic algorithm (GA) of Díaz et al. (2020), the enhanced swarm artificial bee colony ESABC (Díaz et al., 2023a) and the filtered elitist artificial bee colony fEABC (Díaz et al., 2023b), the strongest of the three. Their figures are quoted from those papers, not re-run here.

6 Results

6.1 In-size performance and budget scaling

This section reports what the policy achieves on the class it is trained on, first during training and then at deployment on the validation instances.

Figure 2 shows the training dynamics at the operating budget. Within each block the episode RE falls steeply over the first $\sim$200 episodes and stabilizes, with no divergence in any seed; across blocks, each instance starts better than its predecessor did, the visible effect of carrying the weights over, and the spread between seeds narrows as training advances. The levels in this figure are higher than the deployed ones because training episodes are *sampled*, with exploration active, rather than greedy.

Deployment on the validation instances is reported in Table 7, per instance and for increasing training budgets over ten independent training runs, with Figure 3 showing their means against the budget. Three observations follow. (i) The policy scales monotonically with budget (27.4% $\rightarrow$ 17.4% $\rightarrow$ 13.8% mean RE at 100/300/1000 episodes per instance), with the return per tripling of the budget falling from 10.0 points to 3.6. (ii) The spread between training runs contracts as the budget grows, the standard deviation across the ten seed means falling from 4.69 points at 100 episodes to 1.30 at 300 and 0.47 at 1000, a factor of ten, so what the extra budget yields is not only a better mean but a more reproducible one; this is the deployed counterpart of the narrowing seed band of Figure 2. (iii) The gap to the strongest constructive baseline is a factor $\approx$2.0. Figure 3 places the curve between two dotted references measured on the same six instances: G&T-MWKR above, at 27.9%, and fEABC below, at 9.6%, the latter a reference from per-instance search.

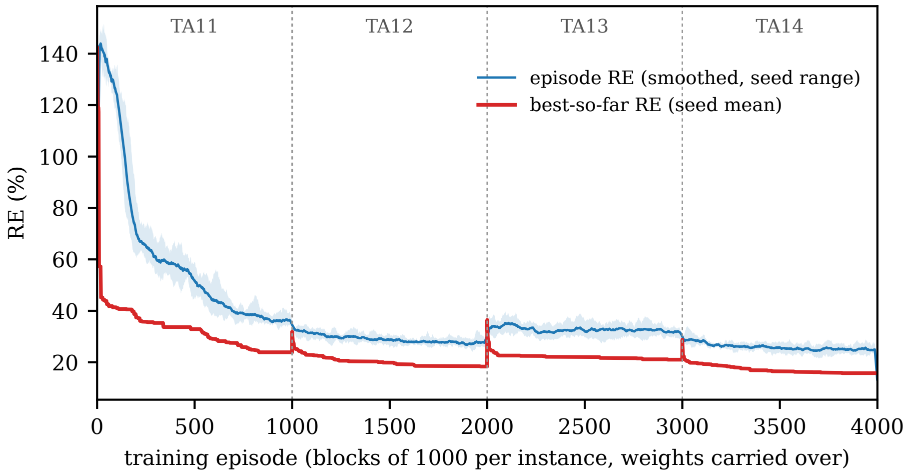

Fig. 2 Training dynamics of the deployed arm: RE of the sampled training episodes (smoothed over 25 episodes; the band spans the seeds) and of the best schedule found so far, over the four training blocks TA11–TA14 with weights carried from block to block. Curves aggregate the seven seeds of the ten whose runs recorded per-episode logs; the remaining three predate that logging.

Table 7 RE (%) per validation instance, mean [best] over ten training runs differing only in their random seed, Deep Sets model, best-of-64 inference. The rightmost column is the standard deviation across the ten per-seed means over the six instances; the last row is the strongest constructive baseline, a single deterministic pass.

Budget	TA15	TA16	TA17	TA18	TA19	TA20	Mean	SD
100 eps	30.2 [21.9]	26.3 [19.7]	26.8 [18.1]	26.7 [18.4]	28.7 [21.5]	25.8 [18.5]	27.4	4.69
300 eps	19.0 [16.3]	16.3 [14.3]	16.6 [13.3]	18.5 [16.2]	17.2 [13.3]	16.6 [14.9]	17.4	1.30
1000 eps	**14.9** [12.1]	**13.2** [11.8]	**12.9** [10.4]	**16.3** [14.6]	**12.8** [11.1]	**12.8** [10.5]	**13.8**	**0.47**
G&T-MWKR	30.6	34.0	17.7	31.8	29.3	24.1	27.9	—

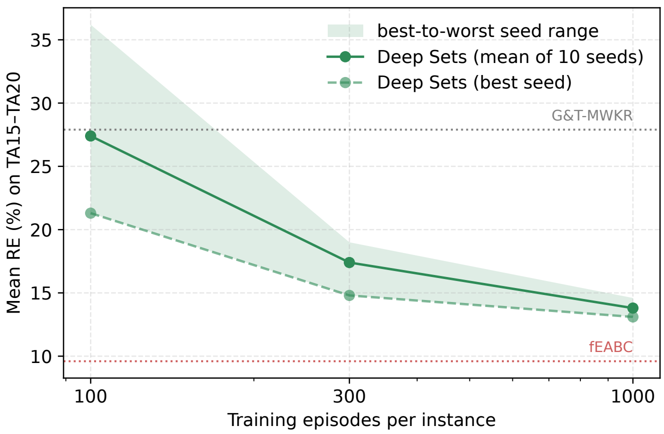

Fig. 3 Mean RE on the six validation instances TA15–TA20 (20×15) against the *training* budget, the episodes spent once per instance before deployment, on a logarithmic axis and over ten training runs. The band spans the best and worst of them; the dotted lines are G&T-MWKR and fEABC on the same six instances.

6.2 Zero-shot cross-size generalization and position among constructive methods

The size-invariant design allows direct evaluation of the 20×15-trained network on any class. For the six classes other than 20×15 the evaluation is *zero-shot*: the trained weights are applied to sizes they never saw, with no additional training or fine-tuning of any kind. The 20×15 class is the exception, since it contains the four training and the six validation instances, and is marked accordingly in the table. Table 8 reports RE over all seventy instances by size class, with its per-instance values in the appendix; the evolved rule of Díaz Rodríguez (2026b) appears beside the policy at each budget, deterministic at one pass and randomized by ϵ-greedy dispatching when sampled, and fEABC closes the table as a 30-run average from a different computational class. Figure 5 shows the two learners' per-instance distributions within each class at the paired budgets.

Every seed's network at 64 samples outperforms G&T-MWKR, the strongest of the hand-crafted baselines considered here, on *every* one of the seventy (210 instance–seed pairs without an exception), with graceful degradation (class means from 11.0% on 15×15 to 22.0% on the hardest transfer, 30×20) and no failure mode up to 1000-operation episodes. Degradation is not monotone in instance size: 50×15, at 12.2%, transfers better than either 30-job class, which the dispatching rules also find easier, so the difficulty the network inherits is the instances' and not a limit of the transfer.

Against the GP rule the comparison is not one-sided: its outcome depends on the inference budget both methods receive (Figure 4). With one pass each, the rule is better. It averages 17.7% against the policy's 19.4%, and the policy improves on it in only 21 of the 70 instances (median deficit 2.0 points, $p = 4.6 \times 10^{-4}$). This comparison is asymmetric, however: the 17.7% rule is the best of 30 independent evolutions, selected on these same 70 instances, while the 19.4% is a mean over the policy's three seeds. Against the mean of the 30 evolutions, 18.99%, the deficit at one pass is 0.4 points. The selected rule is nonetheless retained for the comparison, as the published artifact and the more demanding reference. With 1024 samples each, the GP rule randomized by the ϵ-greedy dispatching its study provides for this purpose, the ordering reverses: 13.0% against 14.1%, with the policy better on 46 of the 70 (median advantage 1.3 points, $p = 1.4 \times 10^{-3}$).

The network's advantage therefore lies in its distribution, whose samples improve faster with budget than those of the randomized rule. The best-of-N ablation of Section 7.2 reaches the same conclusion from the training side. The policy's standalone RE (13.8% in-size, 11–22% zero-shot class means) places it between the dispatching rules (42.3% for EST, the strongest) and the per-instance metaheuristics (9.4% for fEABC). This is the ordering that Zhang et al. (2020) established for the deterministic JSP, in which a learned constructive policy improves on the hand-crafted rules but does not reach the methods that search each instance anew, trading some solution quality for a cost incurred once at training rather than at every instance; it had not been established for the interval variant.

Table 8 Mean RE (%) per size class over the 70 interval Taillard instances, ten per class. The 20×15 column (†) is the training and validation class.

Method	15×15	20×15†	20×20	30×15	30×20	50×15	50×20	All
EST	32.3	49.2	41.5	42.4	56.6	33.2	41.0	42.3
MOR	41.4	46.7	47.3	44.1	58.8	34.1	45.9	45.5
G&T-MWKR	26.7	29.0	30.9	33.6	36.8	23.9	25.5	29.5
GP rule, one pass	15.7	16.6	18.1	21.1	24.1	13.8	14.6	17.7
Policy, greedy	16.9	18.3	18.5	21.2	26.1	15.8	18.8	19.4
GP rule, 64 samples	12.6	15.9	16.8	17.4	23.4	11.7	13.4	15.9
Policy, 64 samples	11.0	13.6	14.8	16.8	22.0	12.2	15.5	15.1
GP rule, 1024 samples	10.2	14.0	15.1	15.5	21.0	10.6	12.1	14.1
Policy, 1024 samples	8.8	11.2	12.9	15.0	19.6	10.4	13.4	13.0
fEABC (30 runs)	5.9	8.7	10.3	9.8	16.4	5.7	9.0	9.4

6.3 Cross-family generalization: the classical instances

The IJSP literature also reports on interval versions of twelve classical instances (FT10 and FT20, seven of the La family, and ABZ7–9), generated from their crisp counterparts with the same symmetric scheme as the Taillard set (Díaz et al., 2020). None of these families was seen during training or validation, so they double as a cross-family generalization test. Following the literature, RE is measured here against the best-known expected-makespan values (Díaz et al., 2023b).

Table 9 RE (%) on the 12 classical interval instances. Column groups are inference budgets: GP and the policy (Pol.) are comparable within a group and not across groups. Each entry is the mean over that method's independent runs, with the per-instance best over those runs in brackets. Reference bounds are best-known expected makespans, not Taillard's crisp bounds.

	rule 1 pass	learned one pass		learned 64 samples		learned 1024 samples		search 30 runs
Inst.	G&T	GP mn [bst]	**Pol.** mn [bst]	GP mn [bst]	**Pol.** mn [bst]	GP mn [bst]	**Pol.** mn [bst]	GA mn [bst]
FT10	32.2	17.0 [7.2]	16.7 [16.0]	11.8 [5.2]	8.5 [6.3]	8.5 [4.2]	6.1 [5.4]	5.2 [1.8]
FT20	40.0	18.5 [9.4]	8.6 [4.4]	13.3 [5.8]	5.8 [4.4]	10.3 [5.4]	4.4 [3.6]	4.4 [1.5]
La21	24.1	15.9 [10.3]	18.3 [15.5]	12.6 [10.0]	12.6 [10.4]	10.7 [6.8]	10.8 [10.7]	5.0 [3.2]
La24	26.3	16.1 [10.1]	19.3 [17.1]	11.5 [8.6]	11.9 [10.9]	9.2 [6.2]	9.1 [8.1]	6.3 [4.1]
La25	16.9	17.6 [9.6]	17.3 [16.7]	12.3 [8.7]	9.6 [9.2]	9.3 [5.1]	8.9 [8.2]	5.1 [1.9]
La27	34.8	18.3 [12.3]	15.0 [13.8]	12.5 [10.7]	11.3 [10.9]	11.3 [8.2]	9.0 [8.1]	10.2 [4.6]
La29	24.5	19.9 [12.9]	21.1 [16.2]	15.0 [11.4]	16.6 [14.0]	12.6 [9.7]	13.3 [12.9]	14.2 [11.1]
La38	27.0	17.4 [9.2]	17.2 [16.6]	14.3 [9.2]	12.8 [11.7]	12.0 [7.1]	9.6 [8.3]	9.2 [6.0]
La40	27.9	16.7 [11.7]	12.0 [9.3]	12.3 [8.8]	9.3 [8.1]	10.0 [7.1]	7.2 [6.5]	8.7 [5.1]
ABZ7	28.7	17.6 [13.3]	16.7 [15.9]	14.3 [11.7]	13.4 [12.0]	12.2 [8.9]	11.9 [11.1]	12.5 [6.3]
ABZ8	36.9	24.6 [18.1]	21.8 [19.1]	20.9 [17.0]	17.3 [16.9]	18.6 [15.0]	15.8 [15.6]	18.5 [11.3]
ABZ9	41.8	29.0 [19.5]	22.6 [19.7]	24.0 [17.9]	19.2 [18.8]	21.5 [17.4]	17.1 [16.6]	18.0 [13.0]
Mean	30.1	19.0 [12.0]	17.2 [15.0]	14.6 [10.4]	12.4 [11.1]	12.2 [8.4]	10.3 [9.6]	9.8 [5.8]

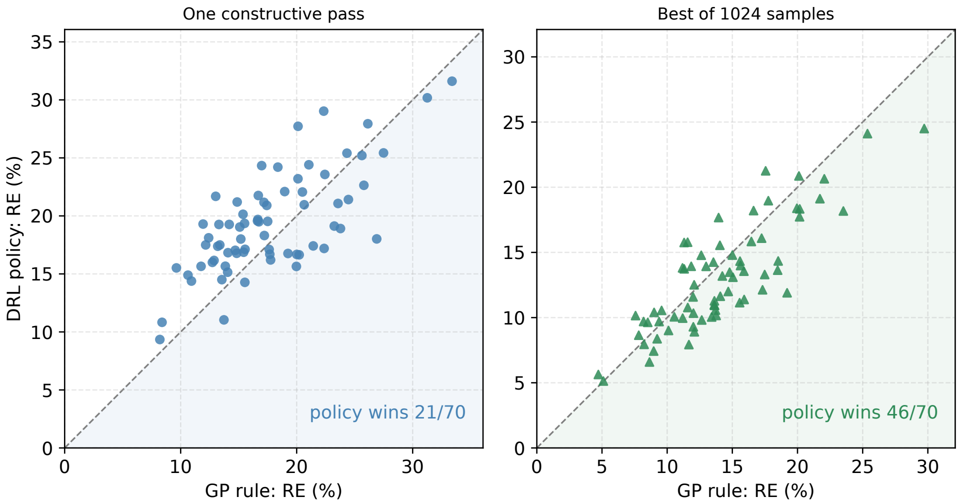

Fig. 4 The DRL policy against the GP rule (Díaz Rodríguez, 2026b) at matched inference budgets, one point per instance over the 70 interval Taillard instances. Points below the diagonal are instances on which the policy is better. Left: one constructive pass each. Right: 1024 samples each, the rule randomized with ϵ-greedy dispatching.

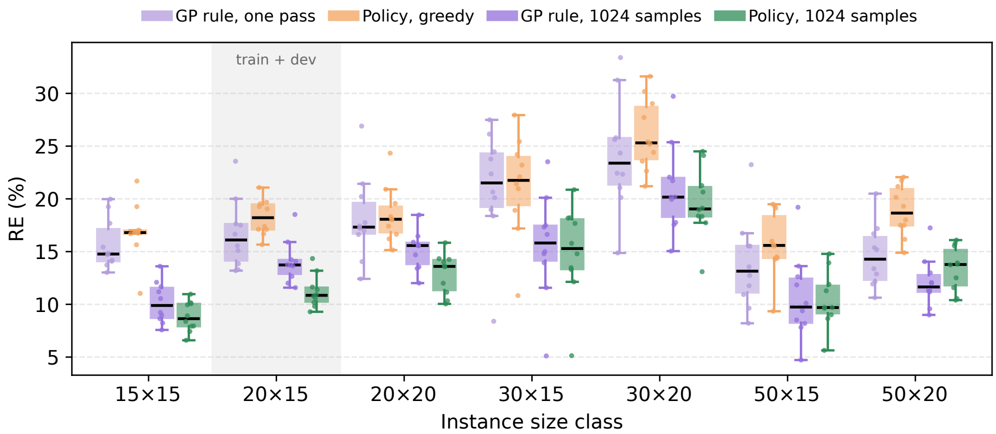

Fig. 5 RE by size class over the 70 interval Taillard instances, the two learners paired at one pass and at 1024 samples. Boxes span the quartiles with whiskers at 1.5 IQR; every instance is overlaid as a point. The shaded 20×15 class holds the training and validation instances.

Table 9 positions the policy (best-of-64, mean and best of the three seeds) against the shared-evaluator rules and the published 30-run averages of the per-instance meta-heuristics, the latter quoted from Table 2 of Díaz et al. (2023b). Its GP columns are the 30 evolutions themselves, the full set being available only on this family, whereas the single featured rule of Section 6.2 represents GP on the Taillard set. The policy improves on EST and G&T-MWKR on all twelve instances and lands at 12.4% mean

RE, between the hand-crafted constructive baselines (30.1% for G&T-MWKR) and the per-instance searchers (GA at 9.8%, ESABC at 5.5%). On La29, La40, ABZ7 and ABZ8 the policy's best seed reaches a lower RE than the 30-run average of GA, a method that searches each of these instances from scratch, whereas the policy has never seen them.

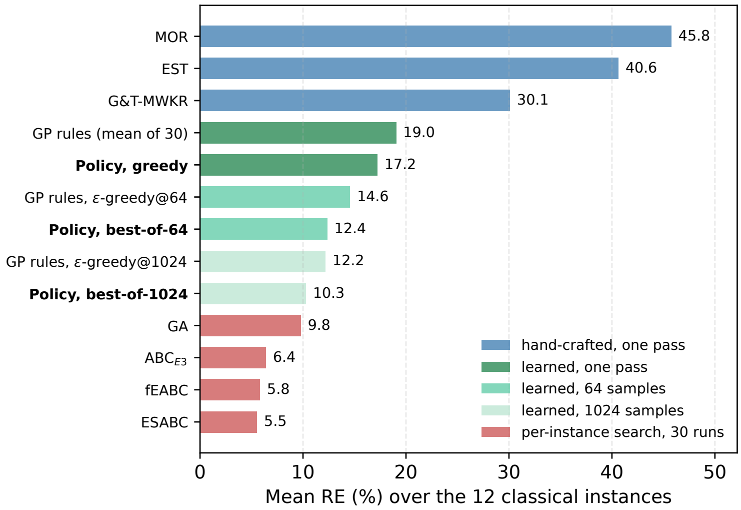

Fig. 6 Mean RE over the 12 classical interval instances, grouped by the inference budget each method is given. Both learned entries are means over independent runs.

Against the evolved rules the comparison must be made budget by budget, as it was on the Taillard set, and Figure 6 is arranged that way. Given one pass each, the policy's 17.2% leads the 19.0% a single evolution should be expected to yield, and trails the featured rule's 17.9% on 5 of the 12 instances (Wilcoxon signed-rank against the featured rule, $p = 0.73$: the two are indistinguishable). Given 64 samples each, with the rules randomized by the ϵ-greedy dispatching their own study provides, the policy leads on both readings, 12.4% against 14.6% for the mean of 30 and 14.2% for the featured rule, better than the latter on 9 of 12, though on twelve instances even that is only marginal ($p = 0.077$). At 1024 samples per run the gap is both the widest and the only one that separates: 10.3% against 12.2%, better on 10 of the 12 instances ($p = 0.009$). Reading these three rows in order, the policy is level with the rules at one pass and separates from them as both are allowed to sample, the same ordering that Section 6.2 reports on the Taillard set, obtained here on a family neither method was tuned for.

One feature of the table requires explanation: the two readings order the learners oppositely. The policy has the better mean at all three budgets, and GP the better bracket at all three. The cause is the number of runs each bracket selects over, 30 evolutions for GP against three training seeds for the policy, compounded by the wider dispersion of the evolved rules: their standard deviation across the 30 averages 4.3, 2.3

and 2.0 points at the three budgets, against 2.2, 1.4 and 0.7 across the policy's seeds. A per-instance minimum taken from 30 draws of a wide distribution reaches further down than one taken from three draws of a narrow one. Quantitatively, the minimum of n normal draws is expected about 0.85 standard deviations below the mean for $n = 3$ and 2.04 for $n = 30$; selecting the policy over 30 seeds would therefore place its bracket near 12.7%, 9.6% and 8.8%, level with the 12.0%, 10.4% and 8.4% of GP instead of behind them. The bracket ordering measures how many runs each side was selected from, not which learner produces the better run. The dispersion across runs is not an artifact, however, and is smaller for the policy at every budget: evolving one rule and training one network do not expose a practitioner to the same risk from the run obtained.

7 Analysis

This section reports which inputs the network consults, which of the design choices earn their cost, and how far executed makespans deviate from the predicted values.

7.1 What the policy attends to

The policy is not interpretable the way a priority expression is, but its inputs can be audited. Figure 7 reports a permutation test (Breiman, 2001): at every decision of a greedy rollout, one feature column is shuffled across the eligible operations, and the resulting mean RE over the validation set and the three seeds is compared with the intact rollouts (18.2%).

Each column is permuted five times with independent draws, and the figure reports the mean degradation and its standard deviation over the five; a single draw moves the magnitudes by several points and reorders the lower half, though not the leading three. Three features carry the policy: the relative slack (whose permutation adds 56.5 ± 2.4 points of RE), the number of remaining operations in the job ($+33.6 \pm 1.4$), and the job's worst-case remaining work ($+9.6 \pm 1.0$). These are the attribute families behind MOR and MWKR, and the very attributes the evolved expressions of Díaz Rodríguez (2026b) select. Below a second tier of three features costing between 1.8 and 2.9 points, the remaining ten fall into a floor between +0.3 and +1.0 points that no measurement of this precision separates, and the two interval-width features sit inside it, one of them last of the sixteen ($+0.3 \pm 0.3$ for the duration width, $+0.9 \pm 0.5$ for the start width). That the widths lie on that floor follows from the objective: the primary key of the criterion of Eq. (3), the worst-case makespan, on which the reward also trains, is a function of the upper endpoints alone (Section 7.4), so a width cannot inform a decision judged by that criterion. The same asymmetry appears within the features themselves. Three quantities enter as a pair of interval endpoints, and in the two pairs that rise above the floor the upper endpoint costs several times its lower counterpart (9.6 against 1.8 for the job's remaining work, 2.1 against 1.0 for the duration), as a criterion written in upper endpoints alone should induce. The floor is not exactly zero because permuting any column perturbs an input distribution the network was trained against, whether or not that column informs the decision.

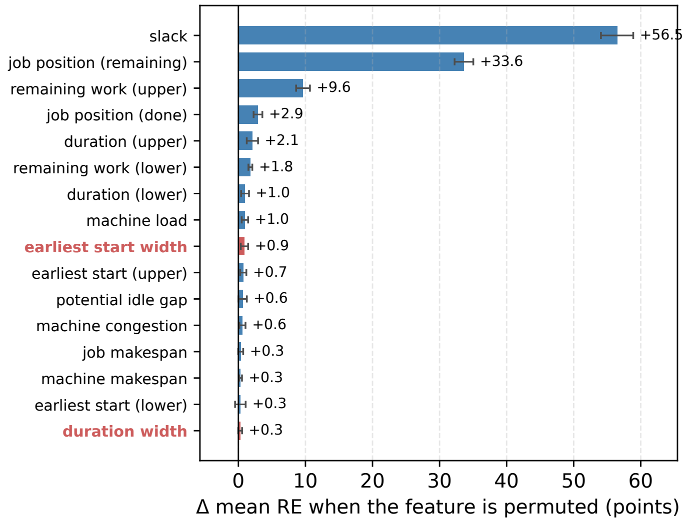

Fig. 7 Permutation importance of the 16 operation features: change in mean validation-set RE when the feature's column is shuffled across the eligible operations at every decision (greedy rollouts, three seeds; intact baseline 18.2%). Bars are the mean of five independent permutation draws and whiskers their standard deviation. The two interval-width features are highlighted.

The audit is therefore conditional on what the policy was trained to optimize, namely this objective and the reward weights of Section 4.1, and a different one need not produce the same ranking. The corresponding test is an objective that does contain the width, which Section 7.4 carries out: retraining under f_λ leaves the weights indistinguishable from the default arm's under a common selection criterion, which is evidence that the ranking would survive that change, though the audit itself was not repeated on those arms.

7.2 Ablation: best-of-N inference

The policy is reported at three inference budgets (one greedy pass, 64 samples and 1024), which leaves its behaviour between them unspecified. To characterize it, all 342 rollouts per checkpoint were retained on each of the 70 instances (71,820 schedules), so that best-of-B is reconstructible for arbitrary B without further evaluation. Figure 8 plots the resulting curve; its band is a dispersion over repetitions, not a confidence interval on the mean. The budget is partitioned over the three checkpoints as in the deployed protocol and the greedy reference is aggregated identically, which places it below the 19.4% of Section 6.2; EST, at 42.3%, lies above the frame. A single *sampled* rollout is substantially worse than the greedy one, 21.4% against 17.0%: the argmax is a high-quality sequence within a broadly dispersed distribution. The curve overtakes the evolved rule at B=6 and the greedy pass at B=12, so sampling becomes profitable at budgets far below the 64 used throughout. And the returns are near-logarithmic over three orders of magnitude, 14.9% at 64 and 12.9% at 1023 (341 per checkpoint, the reconstruction's closest point to the nominal 1024), without an inflection: no budget within the tested range saturates.

The same reconstruction quantifies the marginal return of further sampling and its computational cost. Table 10 reports the mean RE at successive doublings together with the increment each contributes. The marginal return decays slowly but does not vanish, from 1.03 points between 4 and 8 samples to 0.42 between 512 and 1023, a log-linear rate of 0.46 points per doubling over the last three octaves; wall-clock cost, by contrast, scales linearly in B. On the machine of Table 4 a sampled rollout requires 1.03 s on a 20×15 instance and 6.4 s on a 50×20 one, so $B{=}1023$ amounts to 18 minutes and 1.8 hours per instance, and the four additional doublings needed to reach 11% at the measured rate would amount to 4.7 and 29 hours. Sampling is therefore an inefficient route to the residual gap against fEABC's 9.4%, which a per-instance metaheuristic attains in minutes. One qualification bounds this extrapolation: budgets above 1023 lie outside the measured range, and within it the marginal return is itself contracting, with each octave yielding approximately a tenth less than its predecessor, so a constant rate is the optimistic reading.

Table 10 Mean RE over the 70 instances at successive doublings of the inference budget. The second row is how much each budget improves on the budget half its size. Intermediate budgets, omitted for space, follow the same pattern.

Budget B	1	8	64	128	256	512	1023
Mean RE (%)	21.4	17.1	14.9	14.3	13.8	13.3	12.9
Gain over $B/2$ (points)	—	1.03	0.62	0.57	0.50	0.46	0.42

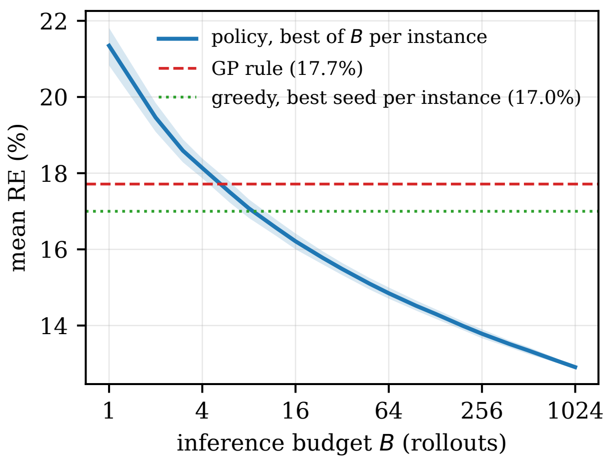

Fig. 8 RE against the *inference* budget B on a base-2 logarithmic axis: for each instance the best of the B samples spent on it is kept, and the curve reports the mean of that value over the 70 instances. The band is the 10–90 percentile over resamplings.

7.3 Ablation: the attention variant

The variant of Section 4.4 inserts two residual attention blocks between the shared per-operation encoder and the masked pooling of Section 4.3, which it leaves in place; everything else is held fixed, including the training and evaluation protocols, the same four training instances, the same six validation instances, the same seeds and the hyperparameters of Table 5. The comparison is therefore paired at the level of (instance, seed), and was conducted at both training budgets employed in this study: over three seeds at 300 episodes and over ten at 1000, the operating budget.

The variant yields no improvement at either budget and a significant degradation at the operating one. Across the eighteen instance–seed pairs it is 1.03 points of RE inferior at 300 episodes, degraded on 12 of the 18 pairs and on 5 of the 6 instances, a difference that does not reach significance (Wilcoxon signed-rank, $p = 0.067$); at 1000 episodes the deficit is 1.06 points over the sixty instance–seed pairs, degraded on 41 of the 60, on all six instances and in every one of the ten seeds ($p < 0.001$). Table 11 reports the corresponding means and best seeds. The variant carries 3.2 times the parameters of the base network and 2.2 times its wall-clock cost per episode, the latter measured at the 300-episode budget, where the three seeds of each arm agree to within 4%.

Table 11 Attention ablation: mean [best seed] RE (%) on the validation set, over three seeds at 300 episodes and ten at 1000.

Configuration	300 eps	1000 eps
Deep Sets	**16.6** [16.2]	**13.8** [13.1]
+ attention (B=2)	17.6 [17.2]	14.9 [14.2]

The representation accounts for part of this outcome. One of the per-operation features is itself relational with respect to the eligible set: slack measures the candidate's earliest start against the earliest among all candidates, so the comparison an attention layer would have to learn along that axis is already available as a precomputed input. A second, machine congestion, is relational with respect to the machines rather than the candidates, contrasting a machine's remaining load with the average over machines. What remains are the pairwise relations these aggregates do not summarize, and at the budgets tested they do not amortize their cost; larger training budgets or more heterogeneous instance distributions may be required for such structure to become profitable.

7.4 What interval awareness contributes

Componentwise, the upper bound of a completion obeys

$$c_o^U = \max\left(c_{JP(o)}^U, c_{MP(o)}^U\right) + p_o^U, \tag{9}$$

a recursion in upper bounds alone: $C^U_{\max}$ is a deterministic function of the processing order and the upper endpoints, and the lower endpoints do not appear in it. $C^U_{\max}$ is also the primary key of the lexicographic criterion of Eq. (3) under which every schedule in this paper is compared. Its secondary key, the lower endpoint, is consulted only on exact ties of the upper one, and on the rollout pools of this section such ties do not arise (the upper-bound minimizer is unique in every one of the 90 (arm, instance, seed) pools), so in practice the criterion reduces to its primary key. The reward of Section 4.1 trains on that primary key, and across its six components not one reads a lower bound, so there is no channel through which knowing an interval's width could reduce what the policy is being trained to reduce. A width feature can at best be an auxiliary signal for learning; it cannot be information about the objective, because the objective does not contain it.

The permutation audit of Section 7.1 found both width features at the floor of its ranking, consistent with this entailment. Three training-time ablations ask the causal questions the audit cannot: whether the policy would be any worse had it never seen the uncertainty at all; whether the upper bounds shown during training must be the true ones, which Eq. (9) does not settle, since midpoint training changes the upper endpoints themselves; and whether the policy would come to consult the widths under an objective that pays for them. Each ablation is paired against the main arm on the training seeds it shares: the first two over all ten seeds, whose baseline is the 13.82% of Section 6.1, and the third over the three seeds of its rollout deposits, whose baseline is the 13.44% mean of those three.

Widths as inputs.

The first ablation removes the information: the encoder collapses every interval to its worst case before the features are computed, so the width features are identically zero and the bounds coincide, while the architecture, the hyperparameters and the schedule builder remain untouched and the environment still executes interval semantics. Ten seeds retrained from scratch reach 13.86% mean RE against the ten-seed baseline's 13.82%: a difference of 0.04 points, worse on 31 of the sixty instance–seed pairs, not significant (Wilcoxon signed-rank, $p = 0.93$). The measurement is not redundant with the prediction, because Eq. (9) concerns the objective while the widths could in principle have shaped the value function or the exploration as an auxiliary signal. They do not, and the effect size bounds what room remains: the paired differences have a standard deviation of 1.86 points, so 0.04 is 0.02 of one, and detecting an effect that small at conventional power would require on the order of three thousand seeds.

Intervals as training data.

The second ablation removes the uncertainty from training altogether. The benchmark perturbs each crisp duration symmetrically, so the original crisp instances *are* the midpoint scenario of their interval counterparts (the coincidence noted after Eq. (4)): the policy is trained on them and evaluated, unchanged, on the interval instances. This is the case Eq. (9) leaves open, since midpoint training replaces the upper endpoints themselves, and both arms extend to ten seeds: the gap is 0.63 points, from 13.82% to 14.45%, worse on 36 of the 60 instance–seed pairs and positive for eight of the ten seeds ($p = 0.045$). Training on the true upper bounds is slightly better than training on their midpoints, a detected effect rather than a measured one, at a power of about 0.7. The delimitation is therefore sharper than a blanket statement about interval awareness. As *input features* the widths are inert, and Eq. (9) says why in a way that generalizes past this network: any method that ranks semiactive schedules by the criterion of Eq. (3) optimizes, through its primary key, a function of the upper endpoints alone, whatever the model class. As *training distribution* the intervals are not inert. The boundary of the claim is the reward and the data, not the architecture.

A width-penalizing objective.

The third ablation changes the objective so that the width enters it. The policy is retrained, unchanged in every other respect, under

$$f_\lambda \;=\; C^U_{\max} + \lambda\,\big(C^U_{\max} - C^L_{\max}\big), \qquad \lambda = 1, \tag{10}$$

which charges the schedule for the width of the interval it predicts and is the analogue of the robust objective of the GP study (Díaz Rodríguez, 2026b); since optimizing one quantity and selecting by another would be incoherent, the best-of-N inference of this arm ranks its samples by f_λ as well. Deployed as that package, the objective delivers what it promises: over the same eighteen instance–seed pairs the predicted interval narrows from 12.85% to 11.90% of the expected makespan (narrower on 13 of 18, $p = 0.010$) while the expected makespan rises by 1.94 points, to 15.38% ($p = 0.007$). Unlike the two ablations above, both effects are significant: a width-rewarding objective is available, at a price of about two points of RE.

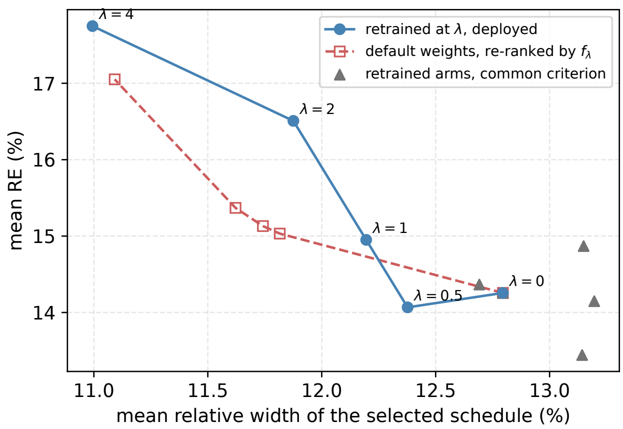

Fig. 9 Mean relative width of the selected schedule against its mean RE, over the 18 (instance, seed) pools of each arm's common rollout deposit. The deployed arms trace the frontier; re-ranking the default policy's deposit reproduces it; under a common criterion the retrained arms do not separate.

Figure 9 locates where that trade is produced. Training and selection changed together in the deployed package, so the two are separated on fresh deposits of 64 rollouts per (arm, instance, seed), with three further arms retrained from scratch at $\lambda = 0.5$, 2 and 4 (three seeds each); every selection is recomputed on these common deposits, which is why the λ=1 figures differ slightly from the training-run schedules quoted above. When deployed, with each arm's samples ranked by its own f_λ, the five arms trace a monotone frontier, from 12.80% relative width at the default end to 10.99% at $\lambda = 4$, with the expected makespan climbing by 3.49 points ($p = 0.0002$). Under a common selection criterion the frontier vanishes: no retrained arm proposes narrower rollouts than the default one (at $\lambda = 1$ the widths are 12.80% against 12.69%, $p = 0.93$, and three of the four arms are marginally wider), with no paired width difference approaching significance (smallest $p = 0.37$). Conversely, re-ranking the *default* policy's own deposit by each f_λ in turn retraces the frontier almost point for point, ending at 11.09% width and 17.05% RE where the arm trained on f_4 lands at 10.99% and 17.75%. What narrows the interval is which sample is kept, not which weights produced it: the network is not merely indifferent to the width features in its input; its proposal distribution does not become narrower when the objective penalizes width, and the retrained weights propose samples as wide as the default's. The predictability–performance trade is real, tunable and monotone, and it arises entirely at selection time.

Contrast with the evolved rule.

The GP study runs the same objective at the same λ and reaches the opposite finding, which makes the two studies informative read together. There, penalizing width narrows the schedules only when the rule has width terminals to act through: sweeping λ from 0 to 4 moves the relative width by 1.64 points for rules that have them and by 0.13 points for rules that do not, from which that study concludes that the objective works *through* the representation (Díaz Rodríguez, 2026b). The network of this paper has both the terminals' equivalent and the objective, and still its weights do not move at any λ; what moves is the ranking among its samples. The same reward therefore reaches the decision function of an evolved rule and does not reach the decision function of a trained policy, at least at this training scale. The rate of the trade is comparable on the two sides, about 2.5 points of RE per point of width across that study's sweep and about 1.9 across ours, so what separates the paradigms is not the magnitude of the cost but the stage at which it is incurred. In GP that stage is the decision function: the evolved rule becomes width-averse, and every schedule it constructs carries the bias. In DRL it is the selection: the weights are unchanged, and the cost arises sample by sample, whenever the ranking passes over a rollout with a better expected makespan in favor of a narrower one. The difference is operational at deployment: changing the trade-off of a width-averse rule requires evolving a new rule, whereas the policy's trade-off is a parameter of the ranking, adjustable at inference time over the same samples. Two explanations remain open: that no training signal reaches these features, or that the semiactive constructive setting fixes the width independently of the policy's decisions.

7.5 Executional robustness

The evaluations so far concern the quality of a predicted schedule; this section concerns its reliability. A schedule is committed as a processing order before the durations are known and is eventually executed under one realization of them, and its prediction is useful insofar as the realized makespan stays close to the expected value reported in advance. This is quantified with the field's $\bar{\varepsilon}$ measure (Díaz et al., 2020; Leon, Wu, & Storer, 1994): for a fixed processing order with predicted makespan $\mathbf{C}_{\max}$,

$$\bar{\varepsilon} = \frac{1}{K}\sum_{k=1}^{K}\frac{\left|C_{\max}^{\mathrm{ex},k} - E[\mathbf{C}_{\max}]\right|}{E[\mathbf{C}_{\max}]}, \tag{11}$$

where $C_{\max}^{\mathrm{ex},k}$ is the crisp makespan of executing the order under the k-th realization, durations drawn independently and uniformly within their intervals. $\bar{\varepsilon}$ is thus the mean relative distance between the executed makespan and the reported value, a measure of the calibration of the prediction, in which lower is more faithful. The measurement uses K=1000 common random numbers per instance (identical realizations for every method, drawn from a generator seeded by the instance identifier so that the draws reproduce across runs and machines), so every comparison below is paired realization by realization. The measurement covers the full seventy-instance benchmark.

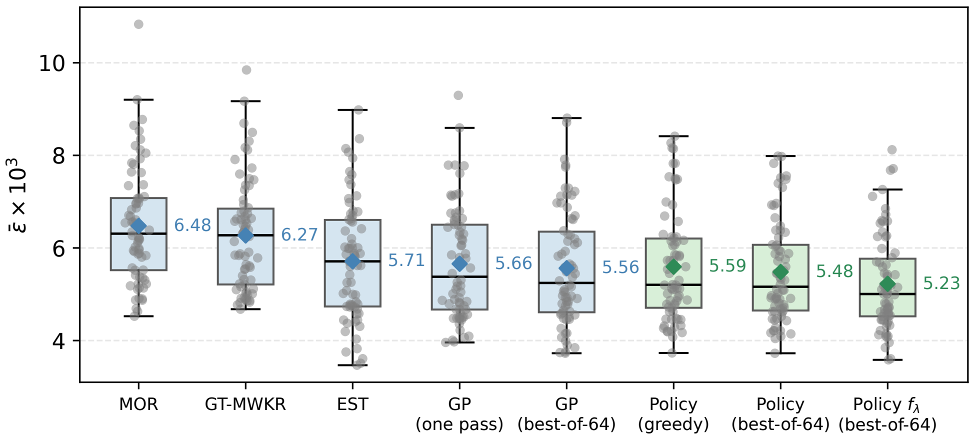

Fig. 10 Executional robustness by method: $\bar{\varepsilon} \times 10^3$ over the seventy benchmark instances; lower is a more faithful prediction. Boxes span the quartiles with whiskers at 1.5 IQR, every instance is overlaid as a point, and diamonds mark the means quoted in the text.

The constructive methods separate into two groups (Figure 10). MOR and G&T-MWKR deviate by $\bar{\varepsilon} \times 10^3 = 6.48$ and 6.27; every remaining method lies between 5.2 and 5.7, and the distance is systematic: the policy at best-of-64 is more faithful than MOR on 63 of the seventy instances and than G&T-MWKR on 65 (paired Wilcoxon signed-rank, $p < 10^{-10}$ in both cases). Within the faithful group the policy's best-of-64 (5.48) is more faithful than EST (5.71; 45/70, $p = 0.011$) and level with the evolved rule of Díaz Rodríguez (2026b) at both of its budgets (5.66 at one pass, $p = 0.069$; 5.56 at 64 samples, $p = 0.38$). Sampling degrades calibration for neither learner: the policy's best-of-64 deviates as its greedy pass does (5.48 against 5.59, $p = 0.29$), and the rule under the ϵ-greedy dispatching of its study as its deterministic pass does (5.56 against 5.66, $p = 0.54$).

The composition of the leading group explains the ranking. It holds a hand-crafted rule and both learners at their two inference budgets, and the presence of EST, the weakest method of this paper in solution quality at 42.3% mean RE (Table 8) yet well clear of MOR and G&T-MWKR in faithfulness, shows that faithfulness and quality are separate properties: a method can predict reliably while scheduling poorly, and optimizing the makespan secures no reliability in return. The mechanism is the interval itself. Every realization's makespan lies within the predicted interval (Section 3), so the width of that interval bounds the deviation any execution can produce: the narrower the prediction, the smaller the largest deviation it admits. None of the methods in the leading group optimizes the width of the interval it predicts, so their faithfulness is bounded by whatever widths their makespan-directed construction happens to produce. Faithfulness must therefore be pursued deliberately rather than inherited from a makespan objective; the GP study reaches the same conclusion from its side, where its rule likewise improves on G&T-MWKR but not on EST until an objective that penalizes width is introduced (Díaz Rodríguez, 2026b).

The robust arm of Section 7.4 is the direct test of that reading: it is the one method in the comparison whose objective charges for the width, and it attains the lowest deviation of the pool, $\bar{\varepsilon} \times 10^3 = 5.23$ against 5.48 for the default arm, more faithful on 49 of the seventy instances ($p < 0.001$) and with the narrowest predictions (11.2% of the expected makespan against 11.8%). This is the direction the argument predicts, demonstrated at benchmark scale, and it comes at the price Section 7.4 measured: about two points of expected makespan. The result completes the chain begun there: the narrowing that Section 7.4 traced to sample selection is not confined to the predicted interval but carries through to execution, as schedules whose realized makespans stay closer to the value reported in advance. Whether that trade is worth making depends on the shop rather than on the model; the comparison above shows it is obtained only when the objective charges for the width.

7.6 Limitations

Four limitations bound the scope of these conclusions. First, the benchmark follows the symmetric ±15% generation scheme standard in the IJSP literature; asymmetric intervals, and distributions in which the uncertainty varies sharply from one operation to another, would separate the worst-case and expected-value criteria far more than they do here, and no policy was trained under them. Second, the policy builds semiactive schedules, so the space it searches excludes the active schedules that the conflict-set restriction reaches and the insertion-based ones beyond them; the strongest constructive baseline of this paper exploits the first of those and the policy still leads it, but the ceiling is real. Third, the λ-sweep arms and the rollout-deposit controls of Section 7.4 are run over three training seeds and compared pairwise on them, so those differences are stated as paired effects over eighteen instance–seed pairs rather than as claims about the population of seeds. The three arms extended to ten seeds calibrate what a three-seed comparison can miss: the midpoint control turned a null into a detected effect, the attention deficit shrank from 1.76 to 1.06 points, and the no-width null sharpened from 0.18 to 0.04 points. Finally, the comparison with the rule evolved by genetic programming shares the evaluation protocol end to end (the same instances, interval arithmetic, ranking criterion, inference budgets and execution realizations, run in a single harness) but not the training side: each learner was trained as its own study prescribes, with budgets that no common currency equalizes, and the featured rule was selected over the full benchmark while the policy's seeds were not. The comparison of the two paradigms with training matched as well, which Xu et al. (2025) call for, remains open.

8 Conclusions and Future Work

The three questions posed in the introduction now have answers. A constructive policy trained by PPO on four interval instances improves on the best hand-crafted constructive baselines by a wide margin on its training class (13.8% versus 27.9% for G&T-MWKR and ≈46% for the best plain rule). Against the rule evolved by genetic programming the answer is not one-sided but budget-dependent: the rule is better at a single pass, and the policy overtakes it when both draw 1024 samples (Section 6.2), so what the network contributes over a compact formula is its sampling distribution rather than its argmax. The per-instance metaheuristics retain the lead in solution quality, on their published figures rather than on a rerun under a common protocol (9.4% mean RE for fEABC against 13.0% for the policy at its largest budget), at the cost of a search on every new instance: the policy occupies the ground between the constructive baselines it dominates and the search algorithms it does not attempt to replace (Q1). Because the representation is dimensionless and the encoder permutation-invariant, one network serves the entire benchmark: evaluated zero-shot at its deployed budget, it outperforms the best dispatching rule on every tested instance of all seven size classes, and the same weights improve on every rule on all twelve classical instances of the FT, La and ABZ families (Q2). Of the standard techniques evaluated, only best-of-N inference improved results at practical budgets: self-attention blocks added to the pooled encoder brought no improvement, and three automatic-configuration campaigns (two over the optimizer, one over the reward weights) failed to beat the hand-set values by more than the resolution of the confirmation protocol (Q3). Three more delimit the interval representation that motivated the design, and they separate two things the design had conflated. As inputs the widths are inert: permuting them costs the policy 0.3 and 0.9 points, the floor of a ranking whose top feature costs 56.5, and removing them from training costs nothing measurable over ten seeds (13.86 against 13.82%, $p = 0.93$), which the closed recursion for the worst-case makespan predicts, since that quantity does not contain a lower bound. As training data the intervals are not inert: fitting the policy to the crisp midpoints instead costs 0.63 points over ten seeds (14.45 against 13.82%,

$p = 0.045$). Retraining under an objective that charges for the width does not overturn this delimitation: swept from $\lambda = 0$ to 4, the deployed arms trace a monotone width–makespan frontier (the predicted interval narrows from 12.80% to 10.99% of the expected makespan, at about 1.9 points of RE per point of width), but with the selection criterion held fixed the robust weights and the default ones are indistinguishable, and ranking the default policy's own samples by f_λ retraces the frontier without any retraining: what narrows the interval is which sample is kept, not which network produced it. At execution time the policy's schedules are among the most dependable of the constructive pool, more faithful than every hand-crafted rule over the seventy instances and level with the evolved rule, while the width-charging arm attains the lowest deviation of the comparison, significantly ahead of the default arm, carrying the selection-time narrowing through to execution. And a permutation audit shows the network leaning on slack, remaining operations and worst-case remaining work, the attributes that hand-crafted and evolved rules exploit, with the width features at the floor of its ranking.

Three directions follow. The first concerns the representation, and the robust arm of Section 7.4 sharpens rather than settles it: paying for the width in the reward narrows the schedules the policy *selects* without measurably changing the policy itself, so the proposal distribution stays as wide when the objective penalizes width as when it does not. Whether the gradient never reaches them, or whether a semiactive constructive decision determines the width before the policy has any say in it, separates two very different diagnoses and calls for a different experiment than any run here: a reward defined on the width alone would tell them apart. The second is deployment: a policy that schedules an instance in seconds is a candidate generator of initial solutions for population-based search, and the quality of its sampling distribution, documented here at a fixed budget, is the property such a use would exploit. The third turns the budget analysis of Section 7.2 inward. The policy commits to a schedule at 17.0% while its own thousand samples reach 12.9%: four points separate the decision the network makes from the best one it produces, and finding that one costs a thousand rollouts. The evolved rule shows the same separation, 17.7% for its single pass against 14.1% randomized to a thousand, so the gap belongs to randomized constructive search rather than to learning as such; what differs is the remedy, since the weights of a network can be moved onto its own best rollouts by retraining on them and iterating, the self-labeling strategy that Corsini, Porrello, Calderara, and Dell'Amico (2024) demonstrate on the crisp JSP, where a formula would have to be evolved anew. How much of the gap is learnable is open, and its shape is measurable: the sampled distribution has a standard deviation of 3.2 points per instance, so its best draw of a thousand sits 2.6 of them below the mean while the argmax sits 1.4; part of that distance is the variance of the tail rather than any decision rule. Such retraining would also sharpen the distribution that best-of-N exploits, so the greedy and the sampled ends have to be measured together.

Appendix A Per-instance results

Table A1 lists, for each of the 70 interval Taillard instances, the crisp lower bound and the RE (%) of the earliest-start rule, of Giffler–Thompson with MWKR tie-breaking, of the evolved rule of Díaz Rodríguez (2026b), and of the policy at its two inference budgets: greedy, averaged over the three training seeds, and best-of-1024. The ten 20×15 instances TA11–TA20 are the training and validation set.

Table A1 Per-instance RE (%) of the constructive methods and of the policy. LB is the crisp Taillard lower bound; Pol. is the greedy policy and Pol.* the best-of-1024 policy.

Inst.	LB	EST	G&T	GP	Pol.	Pol.*	Inst.	LB	EST	G&T	GP	Pol.	Pol.*
TA1	1231	49.9	26.1	19.3	16.8	11.0	TA36	1819	41.1	42.8	23.8	18.9	13.3
TA2	1244	24.9	27.8	14.2	19.3	6.6	TA37	1771	43.4	35.6	18.4	24.2	17.7
TA3	1218	33.0	33.2	14.1	16.8	10.1	TA38	1673	45.0	28.0	20.1	23.2	14.8
TA4	1175	32.0	27.7	13.0	21.7	10.0	TA39	1795	46.6	28.9	22.4	17.2	13.5
TA5	1224	27.2	33.0	14.7	17.0	8.4	TA40	1651	46.9	42.3	26.1	27.9	20.9
TA6	1238	26.8	18.2	13.7	11.0	7.4	TA41	1906	44.6	45.0	33.4	31.6	24.5
TA7	1227	30.7	26.1	17.7	16.7	7.9	TA42	1884	71.1	38.1	24.3	25.4	18.3
TA8	1217	23.7	25.0	14.8	16.8	7.9	TA43	1809	68.7	39.8	22.3	29.0	20.6
TA9	1274	37.7	28.2	20.0	15.6	8.9	TA44	1948	52.1	34.7	25.8	22.6	19.0
TA10	1241	37.0	22.1	15.4	16.9	10.2	TA45	1997	50.2	33.7	14.9	21.2	13.1
TA11	1357	56.2	30.7	17.5	19.5	10.6	TA46	1957	60.8	31.6	22.4	23.6	17.7
TA12	1367	48.8	34.6	17.6	17.1	10.9	TA47	1807	46.8	30.1	25.6	25.2	19.1
TA13	1342	60.2	26.9	13.3	19.3	13.2	TA48	1912	59.1	33.7	21.1	24.4	18.4
TA14	1345	39.5	29.7	13.2	17.4	9.3	TA49	1931	51.0	35.3	20.1	27.7	21.3
TA15	1339	48.1	30.6	16.7	19.7	11.4	TA50	1833	61.4	46.2	31.3	30.2	24.1
TA16	1360	45.2	34.0	15.6	17.1	10.8	TA51	2760	33.8	34.7	23.2	19.1	11.9
TA17	1462	58.9	17.7	13.9	15.7	9.8	TA52	2756	38.0	26.5	12.7	16.0	11.3
TA18	1377	43.8	31.8	23.6	21.1	14.3	TA53	2717	34.1	18.3	11.8	15.7	9.7
TA19	1332	45.8	29.3	20.0	16.7	11.6	TA54	2839	20.9	16.1	8.2	9.3	5.6
TA20	1348	45.7	24.1	15.1	19.0	10.2	TA55	2679	31.7	29.2	15.5	19.4	14.8
TA21	1642	33.4	28.0	20.2	16.6	11.1	TA56	2781	34.8	19.8	10.9	14.4	8.6
TA22	1561	44.2	30.8	17.4	20.9	14.3	TA57	2943	32.9	20.0	9.6	15.5	9.6
TA23	1518	42.8	35.3	17.8	16.2	13.6	TA58	2885	37.4	26.3	13.6	14.5	9.0
TA24	1644	56.3	23.8	14.1	15.1	10.3	TA59	2655	26.6	28.2	16.7	19.5	13.9
TA25	1558	45.8	33.8	16.6	19.6	14.0	TA60	2723	41.7	19.8	15.5	14.3	9.7
TA26	1591	42.8	31.2	26.9	18.0	14.3	TA61	2868	45.7	25.4	15.4	20.1	13.9
TA27	1652	50.0	31.1	21.4	17.4	13.6	TA62	2869	43.0	27.6	17.2	21.2	15.5
TA28	1603	36.8	26.8	12.4	18.1	10.0	TA63	2755	38.8	23.8	13.4	17.5	13.7
TA29	1583	28.8	32.0	17.2	18.3	12.0	TA64	2702	38.2	28.4	12.2	17.5	11.6
TA30	1528	34.2	36.1	17.0	24.3	15.8	TA65	2725	42.9	30.0	20.5	22.1	16.1
TA31	1764	30.7	34.4	24.5	21.4	12.1	TA66	2845	41.8	28.0	11.9	19.3	13.8
TA32	1774	42.7	33.3	19.0	22.1	18.2	TA67	2825	31.7	26.1	15.2	18.0	12.5
TA33	1788	51.3	34.8	27.5	25.4	18.2	TA68	2784	50.1	19.1	10.6	14.9	10.4
TA34	1828	45.9	31.8	20.7	21.0	15.8	TA69	3071	35.3	22.8	12.9	16.2	10.6
TA35	2007	30.3	24.1	8.4	10.8	5.1	TA70	2995	42.9	23.4	16.7	21.8	15.8

Statements and Declarations

Funding.

This work was supported by the Spanish Ministry of Science, Innovation and Universities (MCIN/AEI/10.13039/501100011033) under grant PID2022-141746OB-I00.

Competing interests.

The author has no competing interests to declare that are relevant to the content of this article.

Data availability.

The benchmark instances are openly available in a Zenodo repository (Díaz Rodríguez, 2026a). Code, benchmark harness, all experiment records (accepted and rejected), final checkpoints and figure-generation scripts will be deposited openly upon submission. [TODO: DOI del depósito propio al enviar]

References

Abdullah, S., & Abdolrazzagh-Nezhad, M. (2014). Fuzzy job-shop scheduling problems: A review. *Information Sciences*, *278*, 380–407,

Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S. (2016). *Neural combinatorial optimization with reinforcement learning.* (arXiv:1611.09940)

Bertsimas, D., & Sim, M. (2004). The price of robustness. *Operations Research*, *52*(1), 35–53, https://doi.org/10.1287/opre.1030.0065

Branke, J., Nguyen, S., Pickardt, C.W., Zhang, M. (2016). Automated design of production scheduling heuristics: a review. *IEEE Transactions on Evolutionary Computation*, *20*(1), 110–124, https://doi.org/10.1109/tevc.2015.2429314

Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5–32, https://doi.org/10.1023/A:1010933404324

Corsini, A., Porrello, A., Calderara, S., Dell'Amico, M. (2024). Self-labeling the job shop scheduling problem. *Advances in neural information processing systems (neurips).*

Díaz, H., González-Rodríguez, I., Palacios, J.J., Díaz, I., Vela, C.R. (2020). A genetic approach to the job shop scheduling problem with interval uncertainty. *Information processing and management of uncertainty in knowledge-based systems (IPMU)* (pp. 663–676). Springer.

Díaz, H., Palacios, J.J., Díaz, I., Vela, C.R., González-Rodríguez, I. (2022). Robust schedules for tardiness optimization in job shop with interval uncertainty. *Logic Journal of the IGPL*, *31*(2), 240–254, https://doi.org/10.1093/jigpal/jzac016

Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R. (2023a). An elitist seasonal artificial bee colony algorithm for the interval job shop. *Integrated Computer-Aided Engineering*, *30*(3), 223–242, https://doi.org/10.3233/ica-230705

Díaz, H., Palacios, J.J., González-Rodríguez, I., Vela, C.R. (2023b). Fast elitist ABC for makespan optimisation in interval JSP. *Natural Computing*, *22*(4), 645–657, https://doi.org/10.1007/s11047-023-09953-2

Díaz Rodríguez, H. (2026a). *Genetic programming dispatching rules for the interval job shop: instances, evolved rules, results and code.* Zenodo. ([dataset])

Díaz Rodríguez, H. (2026b). *Genetic programming hyper-heuristics for the job shop scheduling problem with interval durations: robustness-aware interpretable rules that generalize across instance sizes.* (Preprint. [TODO: identificador del preprint])

Gabel, T., & Riedmiller, M. (2008). Adaptive reactive job-shop scheduling with reinforcement learning agents. *International Journal of Information Technology and Intelligent Computing*, *24*(4), 14–18,

Garey, M.R., Johnson, D.S., Sethi, R. (1976). The complexity of flowshop and jobshop scheduling. *Mathematics of Operations Research*, *1*(2), 117–129, https://doi.org/10.1287/moor.1.2.117

Giffler, B., & Thompson, G.L. (1960). Algorithms for solving production-scheduling problems. *Operations Research*, *8*(4), 487–503, https://doi.org/10.1287/opre.8.4.487

González-Rodríguez, I., Vela, C.R., Puente, J. (2010). A genetic solution based on lexicographical goal programming for a multiobjective job shop with uncertainty. *Journal of Intelligent Manufacturing*, *21*(1), 65–73, https://doi.org/10.1007/s10845-008-0161-x

Grumbach, F., Müller, A., Reusch, P., Trojahn, S. (2024). Robust-stable scheduling in dynamic flow shops based on deep reinforcement learning. *Journal of Intelligent Manufacturing*, *35*(2), 667–686, https://doi.org/10.1007/s10845-022-02069-x

Haupt, R. (1989). A survey of priority rule-based scheduling. *OR Spektrum*, *11*(1), 3–16, https://doi.org/10.1007/bf01721162

Karunakaran, D., Mei, Y., Chen, G., Zhang, M. (2016). Dynamic job shop scheduling under uncertainty using genetic programming. *Intelligent and evolutionary systems* (pp. 195–210). Springer.

Kasperski, A., & Zieliński, P. (2016). Robust discrete optimization under discrete and interval uncertainty: A survey. *Robustness analysis in decision aiding, optimization, and analytics.* Springer.

Kayhan, B.M., & Yildiz, G. (2023). Reinforcement learning applications to machine scheduling problems: a comprehensive literature review. *Journal of Intelligent Manufacturing*, *34*, 905–929, https://doi.org/10.1007/s10845-021-01847-3

Kool, W., van Hoof, H., Welling, M. (2019). Attention, learn to solve routing problems! *International conference on learning representations (ICLR).*

Kouvelis, P., & Yu, G. (1997). *Robust discrete optimization and its applications.* Dordrecht: Kluwer Academic Publishers.

Kwon, Y.-D., Choo, J., Kim, B., Yoon, I., Gwon, Y., Min, S. (2020). POMO: Policy optimization with multiple optima for reinforcement learning. *Advances in neural information processing systems (neurips).*

Lei, D. (2011). Population-based neighborhood search for job shop scheduling with interval processing time. *Computers & Industrial Engineering*, *61*(4), 1200–1208, https://doi.org/10.1016/j.cie.2011.07.010

Leon, V.J., Wu, S.D., Storer, R.H. (1994). Robustness measures and robust scheduling for job shops. *IIE Transactions*, *26*(5), 32–43, https://doi.org/10.1080/07408179408966626

López-Ibáñez, M., Dubois-Lacoste, J., Pérez Cáceres, L., Birattari, M., Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, *3*, 43–58, https://doi.org/10.1016/j.orp.2016.09.002

Nazari, M., Oroojlooy, A., Snyder, L., Takáč, M. (2018). Reinforcement learning for solving the vehicle routing problem. *Advances in neural information processing systems (NeurIPS).*

Nguyen, S., Mei, Y., Zhang, M. (2017). Genetic programming for production scheduling: a survey with a unified framework. *Complex & Intelligent Systems*, *3*(1), 41–66, https://doi.org/10.1007/s40747-017-0036-x

Nguyen, S., Zhang, M., Johnston, M., Tan, K.C. (2013). A computational study of representations in genetic programming to evolve dispatching rules for the job shop scheduling problem. *IEEE Transactions on Evolutionary Computation*, *17*(5), 621–639,

Palacios, J.J., González-Rodríguez, I., Vela, C.R., Puente, J. (2015). Coevolutionary makespan optimisation through different ranking methods for the fuzzy flexible job shop. *Fuzzy Sets and Systems*, *278*, 81–97, https://doi.org/10.1016/j.fss.2014.12.003

Panwalkar, S.S., & Iskander, W. (1977). A survey of scheduling rules. *Operations Research*, *25*(1), 45–61, https://doi.org/10.1287/opre.25.1.45

Park, J., Chun, J., Kim, S.H., Kim, Y., Park, J. (2021). Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. *International Journal of Production Research*, *59*(11), 3360–3377, https://doi.org/10.1080/00207543.2020.1870013

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). *Proximal policy optimization algorithms.* (arXiv:1707.06347)

Smit, I.G., Wu, Y., Troubil, P., Zhang, Y., Nuijten, W.P.M. (2024). *Neural combinatorial optimization for stochastic flexible job shop scheduling problems.* (arXiv:2412.14052)

Song, W., Chen, X., Li, Q., Cao, Z. (2023). Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, *19*(2), 1600–1610, https://doi.org/10.1109/TII.2022.3189725

Sotskov, Y.N., Matsveichuk, N.M., Hatsura, V.D. (2020). Schedule execution for two-machine job-shop to minimize makespan with uncertain processing times. *Mathematics*, *8*(8), 1314, https://doi.org/10.3390/math8081314

Taillard, E. (1993). Benchmarks for basic scheduling problems. *European Journal of Operational Research*, *64*(2), 278–285, https://doi.org/10.1016/0377-2217(93)90182-M

Tassel, P., Gebser, M., Schekotihin, K. (2021). *A reinforcement learning environment for job-shop scheduling.* (arXiv:2104.03760)

Đurašević, M., & Jakobović, D. (2018). A survey of dispatching rules for the dynamic unrelated machines environment. *Expert Systems with Applications*, *113*, 555–569, https://doi.org/10.1016/j.eswa.2018.06.053

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., … Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems (NeurIPS).*

Vinyals, O., Fortunato, M., Jaitly, N. (2015). Pointer networks. *Advances in neural information processing systems (NeurIPS).*

Wang, Y., Guo, T., Liu, Z. (2025). *An expectation-maximization algorithm-based autoregressive model for the fuzzy job shop scheduling problem.* (arXiv:2502.00018)

Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., … Liu, T.-Y. (2020). On layer normalization in the transformer architecture. *International conference on machine learning (ICML).*

Xu, M., Mei, Y., Zhang, F., Zhang, M. (2025). Learn to optimise for job shop scheduling: a survey with comparison between genetic programming and reinforcement learning. *Artificial Intelligence Review*, *58*, 160, https://doi.org/10.1007/s10462-024-11059-9

Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R., Smola, A. (2017). Deep sets. *Advances in neural information processing systems (NeurIPS).*

Zhang, C., Cao, Z., Song, W., Wu, Y., Zhang, J. (2024). Deep reinforcement learning guided improvement heuristic for job shop scheduling. *International conference on learning representations (iclr).*

Zhang, C., Song, W., Cao, Z., Zhang, J., Tan, P.S., Xu, C. (2020). Learning to dispatch for job shop scheduling via deep reinforcement learning. *Advances in neural information processing systems (NeurIPS).*

Zheng, P., Xiao, S., Zhang, P., Lv, Y. (2024). A two-individual-based evolutionary algorithm for flexible assembly job shop scheduling problem with uncertain interval processing times. *Applied Sciences*, *14*(22), 10304, https://doi.org/10.3390/app142210304